B.Tech. BCSE497J - Project-I

AI Powered Prediction Model for Urban Traffic Optimization using XGBoost

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

23BDS0187 G. Saketh Reddy

23BDS0245 S. Yaswanth Reddy

23BDS0305 K. Srijan

Under the Supervision of

Dr. RadhaKrishna Mohanty

Assistant Professor Senior Grade 1

School of Computer Science and Engineering (SCOPE)



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

DECLARATION

I hereby declare that the project entitled AI Powered Prediction Model for Urban Traffic Optimization using XGBoost .submitted by me, for the award of the degree of Bachelor of Technology in Computer Science and Engineering to VIT is a record of bonafide work carried out by me under the supervision of Dr. RadhaKrishna Mohanty .

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place : Vellore

Date : 21/10/2026

G. Saketh Reddy
S. Yaswanth Reddy
K. Srijan

Signature of the Candidate

CERTIFICATE

This is to certify that the project entitled “Urban-Shield AI: Urban Traffic Intelligence and Road Safety Analytics Platform” submitted by G. Saketh Reddy (23BDS0187),S. Yaswanth Reddy (23BDS0245), K. Srijan (23BDS0305), School of Computer Science and Engineering, VIT, for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by him / her under my supervision during Fall Semester 2026-2027, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place : Vellore

Date : 21/10/2026

Signature of the Guide

Examiner(s)

Dr. SENDHIL KUMAR K S
Computer Science and Engineering(Data science)

ACKNOWLEDGEMENTS

I am deeply grateful to the management of Vellore Institute of Technology (VIT) for providing me with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in my academic journey. The support and infrastructure provided by VIT have enabled me to explore and develop my ideas to their fullest potential.

My sincere thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering (SCOPE), for his unwavering support and encouragement. His leadership and vision have greatly inspired me to strive for excellence. The Dean's dedication to academic excellence and innovation has been a constant source of motivation for me. I appreciate his efforts in creating an environment that nurtures creativity and critical thinking.

I express my profound appreciation to Dr. SENDHIL KUMAR K S ,the Head of the Computer Science and Engineering(Data science), for his/her insightful guidance and continuous support. His/her expertise and advice have been crucial in shaping the direction of my project. The Head of Department's commitment to fostering a collaborative and supportive atmosphere has greatly enhanced my learning experience.

I am immensely thankful to my project guide, Dr. RadhaKrishna Mohanty , for his/her dedicated mentorship and invaluable feedback. His/her patience, knowledge, and encouragement have been pivotal in the successful completion of this project. My supervisor's willingness to share his/her expertise and provide thoughtful guidance has been instrumental in refining my ideas and methodologies.

Thank you all for your contributions and support.

G. Saketh Reddy
S. Yaswanth Reddy
K. Srijan
Name of the Candidate

TABLE OF CONTENTS

Sl. No.	Contents	Page No.
	Declaration	ii
	Certificate	iii
	Acknowledgements	iv
	List of Figures	viii
	List of Tables	x
	List of Abbreviations	xi
	Symbols and Notations	xii
	Abstract	**xiii**
1.	**INTRODUCTION**	**1**
	1.1 Background	1
	1.2 Motivations	1
	1.3 Scope of the Project	2
2.	**PROJECT DESCRIPTION AND GOALS**	**4**
	2.1 Literature Review	4
	2.1.1 Density-based discovery of accident hotspots	4
	2.1.2 Short-term traffic forecasting	5
	2.1.3 Risk-aware and multi-objective route planning	5
	2.1.4 Explainability	6
	2.2 Research Gap	6
	2.3 Objectives	7
	2.4 Problem Statement	7
	2.5 Project Plan	8
3.	**TECHNICAL SPECIFICATION**	**10**
	3.1 Requirements	10
	3.1.1 Functional Requirements	10
	3.1.2 Non-Functional Requirements	11
	3.2 Feasibility Study	12
	3.2.1 Technical Feasibility	12
	3.2.2 Economic Feasibility	13
	3.2.3 Social Feasibility	13
	3.3 System Specification	14
	3.3.1 Hardware Specification	14
	3.3.2 Software Specification	14

4. DESIGN APPROACH AND DETAILS 17
4.1 System Architecture 17
4.2 Design 18
4.2.1 Data Flow Diagram 18
4.2.2 Use Case Diagram 20
4.2.3 Class Diagram 20
4.2.4 Sequence Diagram 21
5. METHODOLOGY AND TESTING 23
5.1 Development Methodology 23
5.2 Module Description 23
5.2.1 Authentication and Authorisation Module 23
5.2.2 Live Telemetry Module 23
5.2.3 Road Safety Index Module 24
5.2.4 Accident Risk Module 24
5.2.5 Traffic Projection Module 25
5.2.6 Hotspot Detection Module 25
5.2.7 Multi-Objective Routing Module 26
5.2.8 Scenario Simulation Module 27
5.2.9 Conversational Assistant Module 27
5.2.10 Dataset Management and Real-Time Modules 27
5.3 Testing 27
5.3.1 Testing Methodology 27
5.3.2 Automated Test Results 28
5.3.3 Manual Functional Test Cases 28
5.4 Error Handling, Limitations and Edge Cases 30
5.4.1 Defect Identified: Unauthenticated Fallback Identity 30
5.4.2 Dual Data Path in Hotspot Detection 30
5.4.3 Error Handling Strategy 30
5.4.4 Edge Cases Considered 31
6. PROJECT DEMONSTRATION 32
6.1 Authentication and Role Selection 32
6.2 Overview Dashboard 33
6.3 Live Traffic GIS 34
6.4 Traffic Forecast 35
6.5 Accident Intelligence and Hotspot Detection 36
6.6 Road Safety Index 37

6.7 Smart Routing and Emergency Priority 38
6.8 Incident Reporting and Weather Telemetry 39
6.9 City Analytics and Command Centre 40
6.10 Scenario Simulation 41
6.11 Conversational Assistant 42
6.12 Model Registry 42
7.1 Implementation Outcome Against Objectives 43
7.2 Density-Based Hotspot Detection Results 43
7.3 Road Safety Index Verification 45
7.4 Multi-Objective Routing Results 46
7.5 Performance Measurements 47
7.6 Cost Analysis 48
8. CONCLUSION 49
9. REFERENCES 51
APPENDIX A – SAMPLE CODE 55
A.1 Password Derivation and Verification 55
A.2 Role-Based Authorisation Dependency 56
A.3 Identity Resolution and the Fallback Defect 56
A.4 DBSCAN Spatial Hotspot Detection 57
A.5 Transparent Road Safety Index 57
A.6 Accident Risk Estimation with Contributing Factors 58
A.7 Scenario Simulation Engine 58
A.8 Feature Engineering Pipeline for Project-II 59
A.9 Road Segment ORM Entity 59
A.10 WebSocket Broadcast of New Incidents 60
A.11 Client API Layer with Token Propagation and Safe Degradation 60
A.12 Automated Test Case Exposing the Authorisation Defect 60

List of Figures

Figure No.	Title	Page No.
2.1	Project-I development plan showing the seven phases and their overlap	8
4.1	Layered system architecture of Urban-Shield AI	17
4.2	Context-level data flow diagram (DFD Level 0)	18
4.3	Level 1 data flow diagram showing the eight sub-processes and seven data stores	19
4.4	Use case diagram showing four actors and fifteen use cases	20
4.5	Class diagram of the backend application, analytics engines and ORM entities	21
4.6	Sequence diagram for the accident hotspot discovery request	22
6.1	Unified sign-in modal with the four seeded role personas	32
6.2	Overview dashboard with summary tiles, quick what-if controls and the multi-layer GIS panel	33
6.3	Live traffic GIS view with congestion-coloured corridors and overlay controls	34
6.4	Segment inspector opened for a selected corridor	34
6.5	Multi-horizon traffic forecast view	35
6.6	Accident intelligence view over the stored accident corpus	36
6.7	DBSCAN spatial hotspot detection view with per-cluster risk scores and peak risk windows	36
6.8	Road Safety Index view with the published formula, rating bands and corridor leaderboard	37
6.9	Smart routing engine showing the fastest and safest objectives computed for one origin-destination pair	38
6.10	Emergency priority dispatch view	38
6.11	Incident reporting view with crowd-sourced hazard reports	39
6.12	Weather telemetry view with road friction and six-hour outlook	39
6.13	City analytics view with aggregate network indicators	40
6.14	Command centre view with multi-layer spatial control	40
6.15	What-if scenario simulator showing baseline	41

	and simulated outcomes for a combined surge, rainfall and closure scenario	
6.16	Conversational assistant answering a query about currently dangerous corridors	42
6.17	Model registry view listing the declared model inventory and its metric fields	42
7.1	Distribution of cluster sizes produced by DBSCAN over the stored accident corpus	44
7.2	Penalty decomposition of the Road Safety Index for segment 1	45
7.3	Travel time against safety score for the five routing objectives	46

List of Tables

Table No.	Title	Page No.
2.1	Development phases, activities and deliverables	8
3.1	Functional requirements and their implementing components	10
3.2	Non-functional requirements	11
3.3	Indicative annual operating cost estimate for a single-city deployment	13
3.4	Hardware specification	14
3.5	Software environment	14
3.6	Audit of declared backend dependencies against actual source imports	15
5.1	Routing objective parameters as implemented	26
5.2	Automated test suite results (pytest, 8 cases, 7 passed, 1 failed)	28
5.3	Manual functional test cases and observed results	29
5.4	Edge cases and system behaviour	31
7.1	Outcome against stated objectives	43
7.2	DBSCAN clustering results over the stored accident corpus	44
7.3	Verification of the Road Safety Index for segment 1	45
7.4	Routing alternatives returned for one origin-destination pair of 3.72 km	46
7.5	Measured API response times over 30 sequential requests per endpoint	47

List of Abbreviations

ADR	Architecture Decision Record
AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
DFD	Data Flow Diagram
DOT	Department of Transportation
ER	Entity-Relationship
GIS	Geographic Information System
HDBSCAN	Hierarchical Density-Based Spatial Clustering of Applications with Noise
HMAC	Hash-Based Message Authentication Code
HTTP	Hypertext Transfer Protocol
ITS	Intelligent Transportation Systems
JSON	JavaScript Object Notation
JWT	JSON Web Token
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
ML	Machine Learning
MoRTH	Ministry of Road Transport and Highways
ORM	Object-Relational Mapper
PBKDF2	Password-Based Key Derivation Function 2
PostGIS	Spatial Database Extender for PostgreSQL
RBAC	Role-Based Access Control
REST	Representational State Transfer
RMSE	Root Mean Squared Error
SHA	Secure Hash Algorithm
SHAP	SHapley Additive exPlanations
SPA	Single-Page Application
SQL	Structured Query Language
UI	User Interface
VPH	Vehicles Per Hour
VPK	Vehicles Per Kilometre
WHO	World Health Organization
XGBoost	Extreme Gradient Boosting

Symbols and Notations

S	Road Safety Index of a segment, on a 0–100 scale
P	A bounded penalty term contributing to the Road Safety Index
W	Accumulated weighted risk score prior to normalisation
a	Historical accident frequency rating of a segment
c	Current congestion of a segment, per cent
r	Rainfall intensity, millimetres per hour
v	Meteorological visibility, metres
h	Forecast horizon, minutes
$\hat{c}(h)$	Projected congestion at horizon h, per cent
$\hat{v}(h)$	Projected mean speed at horizon h, kilometres per hour
$\hat{q}(h)$	Projected volume at horizon h, vehicles per hour
Eps	DBSCAN neighbourhood radius
MinPts	DBSCAN minimum neighbourhood population
R	Heuristic risk score assigned to a discovered cluster
d	Estimated route distance, kilometres
t	Estimated travel time, minutes
ΔV	Scenario change in traffic volume, per cent
Δ	A delta or change quantity in the scenario simulator

ABSTRACT

Rapid urbanisation has made road congestion and road-traffic injury two of the most persistent problems faced by city administrations. The World Health Organization records approximately 1.19 million road-traffic deaths each year and identifies road crashes as the leading cause of death for people aged 5 to 29 years, while the Ministry of Road Transport and Highways recorded 4,80,583 road accidents and 1,72,890 fatalities in India during 2023. The operational tools available to municipal authorities are typically fragmented: congestion monitoring, accident record-keeping, blackspot analysis and route guidance are delivered by separate systems that do not share a common data model, and the scores they produce are rarely explainable to the officer who must act on them.

This Project-I report presents Urban-Shield AI, a working full-stack urban traffic intelligence and road safety analytics platform that unifies these functions behind a single relational schema and a single API. The system is implemented as a FastAPI service exposing nineteen REST endpoints and one WebSocket channel over a ten-table SQLAlchemy schema, together with a React 18 and TypeScript single-page application providing fifteen analytical views, a Leaflet GIS map and four role-specific dashboards for commuters, emergency responders, city authorities and administrators. Access is controlled by JSON Web Tokens with passwords stored as PBKDF2-HMAC-SHA256 digests over 200,000 iterations, and endpoint authorisation is enforced through a role dependency.

The analytical core comprises six engines. Accident hotspot discovery is performed by density-based spatial clustering using the scikit-learn implementation of DBSCAN with a neighbourhood radius of 350 metres and a minimum of four points. The remaining five engines — multi-horizon traffic projection, accident risk classification, a transparent Road Safety Index, multi-objective route planning and a what-if scenario simulator — are deterministic weighted-formula models whose every term is exposed to the interface, so that each score can be decomposed and explained rather than merely displayed. This design decision is stated explicitly: no supervised model is trained in the present phase, and the performance figures held in the model registry table are seeded placeholders rather than measured results.

Evaluation was carried out against the seeded operational database of 41 road segments, 960 traffic records and 181 accident records. Applying DBSCAN to the accident corpus produced 31 spatial clusters covering 157 points, with 24 points (13.3 per cent) correctly rejected as spatial noise. The Road Safety Index and the scenario simulator were verified by recomputing their published outputs from first principles and reproducing them exactly. The automated test suite executes eight cases, of which seven pass; the single failure exposed a genuine authorisation defect in which an unauthenticated request is silently granted a fallback identity, and this defect is reported rather than concealed. Median API response times measured locally ranged from 5.5 to 21.3 milliseconds. The outcome is a complete, reproducible and honestly characterised platform that provides a sound foundation for the supervised learning work planned for Project-II.

CHAPTER 1

INTRODUCTION

1.1 Background

Urban mobility has become one of the defining infrastructure challenges of the present decade. As metropolitan populations and private vehicle ownership continue to rise, road networks that were dimensioned for an earlier era are routinely operated at or beyond their design capacity. The consequences are measured not only in lost time and fuel, but in human life. The World Health Organization reports approximately 1.19 million road-traffic deaths annually, notes that road crashes remain the leading cause of death among people aged 5 to 29 years, and records that nine in every ten of these deaths occur in low- and middle-income countries [51]. For India specifically, the Ministry of Road Transport and Highways recorded 4,80,583 road accidents, 1,72,890 fatalities and 4,62,825 injuries during the calendar year 2023, an increase of 4.2 per cent in accident count over the preceding year, with over-speeding alone accounting for 68.1 per cent of persons killed under traffic rule violations [52].

Intelligent Transportation Systems have emerged as the principal technical response to this situation. Modern traffic management increasingly depends on the ability to observe the network continuously, to project its state a short distance into the future, to identify the locations at which crash risk concentrates, and to guide vehicles in a way that accounts for risk as well as for travel time. Each of these capabilities has an established research literature. Density-based spatial clustering has become the standard unsupervised technique for discovering accident blackspots from geo-referenced crash records, because it finds clusters of arbitrary shape without requiring the number of clusters to be specified in advance and because it explicitly labels isolated incidents as noise [53], [59], [60]. Gradient-boosted tree ensembles and recurrent neural architectures dominate short-term traffic flow forecasting [54], [55], [61]. Risk-aware route planning, in which a path is selected to minimise exposure to hazardous segments rather than simply to minimise duration, has been demonstrated on large-scale road networks [56].

What has advanced less quickly is the integration of these capabilities into a single operational picture. A traffic control room commonly runs congestion monitoring, accident record-keeping, blackspot studies and public route guidance as separate systems, built at different times by different vendors, over data models that do not correspond. An officer who observes that a corridor has become congested cannot immediately see whether that corridor is also a historical crash cluster, what the projected state of the corridor will be in the next hour, or how a proposed closure would redistribute demand. This project addresses that integration gap.

1.2 Motivations

Three specific observations motivated the design of Urban-Shield AI.

The first is the fragmentation already described. Because congestion, crash history, weather exposure and network geometry are held in unrelated systems, any analysis that depends on more than one of them has to be assembled manually. The project therefore adopts a single relational schema in which road segments, time-series telemetry, accident records, weather observations, user-reported incidents and derived hotspots are all first-class entities related to one another, so that

a cross-cutting question can be answered by a single query path.

The second is the opacity of risk scores. A safety or risk score that is presented to an operator as a single number, with no account of how it was arrived at, is difficult to act upon and difficult to defend. If a corridor is rated as dangerous, the officer responsible needs to know whether that rating is driven by crash history, by present congestion, by weather exposure or by junction geometry, because the remedy differs in each case. This motivated the decision to build the Road Safety Index and the accident risk classifier as transparent additive models in which every penalty or weight is returned alongside the score and rendered in the interface, rather than as opaque predictors.

The third is the narrowness of conventional route guidance. Commercial navigation optimises almost exclusively for travel time. A commuter who would willingly accept a small delay in exchange for avoiding known crash clusters, a municipal fleet operator concerned with emissions, and an ambulance crew requiring signal priority are three different optimisation problems that mainstream tools treat as one. The platform therefore exposes five distinct routing objectives over the same origin-destination pair and presents their trade-offs side by side.

A fourth, pedagogical motivation should also be stated. Building the complete vertical slice — schema, API, authorisation, analytics and interface — rather than a notebook in isolation, exposes the engineering concerns that determine whether an analytical method is usable in practice: how features reach the model at inference time, how results are made explainable, how access is controlled, and how the system behaves when data is missing.

1.3 Scope of the Project

The scope of Project-I is the design and implementation of a complete, working and internally consistent platform, together with an honest characterisation of what it does and does not yet do. The following items are within scope and are implemented in the submitted codebase:

- A normalised relational schema of ten tables covering users, road segments, traffic time series, traffic projections, accident records, hotspots, weather observations, user-reported incidents, a model registry and alerts, expressed through the SQLAlchemy 2.0 object-relational mapper.
- A FastAPI service exposing nineteen REST endpoints and one WebSocket channel, documented automatically through an OpenAPI schema.
- Token-based authentication using JSON Web Tokens, with passwords stored as salted PBKDF2-HMAC-SHA256 digests over 200,000 iterations, and role-based authorisation across four roles.
- Six analytical engines: multi-horizon traffic projection, accident risk classification, density-based hotspot detection using scikit-learn DBSCAN, a transparent Road Safety Index, a five-objective routing engine, and a what-if scenario simulator.
- A React 18 and TypeScript single-page application comprising fifteen analytical views, a Leaflet GIS map with colour-coded segment rendering, Recharts visualisations and four role-specific dashboards.
- A seeded operational dataset of 41 road segments, 960 traffic records, 181 accident records and associated weather and incident data, together with a CSV import path for user-supplied corridors.

- An automated test suite executed with pytest, and containerisation through Docker Compose.

The following items are deliberately outside the scope of the present phase and are identified as Project-II work. Stating them explicitly is part of the contribution, because the distinction between an implemented capability and a declared intention is frequently blurred in prototype systems:

- Training, validation and deployment of supervised learning models. The traffic projection and accident risk engines are deterministic weighted formulations, not fitted estimators. No model artefact is trained or persisted, and the metric fields in the model registry table hold seeded placeholder values that are not measurements.
- Ingestion of live sensor or camera telemetry. The platform operates on a seeded and user-importable dataset; no external traffic or weather feed is connected, and the API keys reserved for such feeds in the configuration module are unused.
- Use of a large language model in the conversational assistant. The assistant is implemented as a deterministic keyword-dispatch function over live database aggregates.
- Deployment of the PostgreSQL and PostGIS backend. Although the data access layer is written against a database-agnostic ORM and the connection string is configurable, all testing reported here was performed against the SQLite backend.
- Load, concurrency and multi-user performance testing. The timing figures reported in Chapter 7 are single-user measurements taken on a local instance.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

The literature relevant to this project falls into four groups: density-based discovery of accident hotspots, short-term traffic forecasting, risk-aware route planning, and the algorithmic foundations on which these rest. Each is reviewed below, and the review is used in Section 2.2 to identify the gap this project addresses. Throughout this section a clear separation is maintained between results published by other researchers and the work carried out in this project; no performance figure reported in this section is claimed for Urban-Shield AI.

2.1.1 Density-based discovery of accident hotspots

Ester, Kriegel, Sander and Xu introduced DBSCAN as a density-based clustering algorithm for large spatial databases [53]. The algorithm requires two parameters, a neighbourhood radius (Eps) and a minimum neighbourhood population (MinPts), and classifies every point as a core point, a border point or noise. Two properties make it particularly suitable for crash data: the number of clusters need not be known in advance, and isolated incidents are explicitly rejected as noise rather than being forced into a cluster. Both properties are directly relevant to blackspot analysis, where the number of true blackspots is unknown and where a single isolated collision should not generate a spurious hotspot.

Subsequent transport-domain studies have confirmed this suitability. Zhang, Zhu, Li, Qiu and Xie applied density clustering to expressway accident records and combined the resulting spatial clusters with a Bayesian network in order to characterise the spatiotemporal patterns underlying each cluster, demonstrating that density-based segmentation provides a meaningful unit of analysis for subsequent causal modelling [59]. Chávez-García, Salinas-Carrión, Navas-Perrone and González-Rodríguez used DBSCAN together with ensemble time-series modelling to analyse and forecast traffic accidents in Ecuador, establishing a two-stage pattern in which spatial clustering first localises risk and temporal modelling then projects it [60]. More recently, Roy, Mohammadi and Roy compared HDBSCAN with conventional spatial clustering techniques for GIS-based hotspot detection, reporting that hierarchical density methods can accommodate clusters of varying density within a single network [46].

The present project adopts the classical DBSCAN formulation of Ester et al. through its scikit-learn implementation [57], with Eps set to 350 metres and MinPts set to four. The choice of the classical algorithm over HDBSCAN is deliberate and is discussed in Section 5.2.6; in brief, a fixed and interpretable neighbourhood radius expressed in metres is easier for a municipal user to reason about than a variable-density hierarchy.

The wider literature situates these methods. GIS-based hotspot mapping and spatial pattern analysis of road accidents form an established strand [41]–[45], and machine learning has since been combined with geographic information systems to detect accident-prone locations and classify hotspot severity [46]–[48], [50]. A parallel body of work models crash frequency and injury severity rather than their spatial concentration: methodological reviews of accident research [31]–[33], [39], studies relating real-time traffic conditions to crash risk [34]–[36], machine learning approaches to severity classification [37], [38], [49], and treatments of unobserved

heterogeneity in severity models [40]. The present project draws on the spatial strand rather than the severity strand, because the operational question it addresses is where risk concentrates across a network rather than how severe an individual crash will be.

2.1.2 Short-term traffic forecasting

Short-term forecasting of flow, speed or congestion has been addressed with both ensemble tree methods and recurrent neural architectures. Chen and Guestrin introduced XGBoost, a scalable gradient-boosting system whose regularised objective and sparsity-aware split finding made gradient-boosted trees practical on large tabular datasets [54]; it has since become a common baseline in traffic prediction. Hochreiter and Schmidhuber's long short-term memory network [55] provides the complementary recurrent approach, able to retain information across long input sequences and therefore well suited to traffic series exhibiting daily and weekly periodicity.

Comparative and hybrid work indicates that the two families are complementary rather than competing. Zhang and Zhang proposed an LSTM-XGBoost combination model for short-term traffic flow prediction, using the recurrent component to capture temporal dependence and the boosted ensemble to model the residual nonlinearity, and reported improved accuracy over either component used alone [61]. A consistent finding across this literature is the importance of feature construction: temporal indicators such as hour of day, day of week and rush-hour membership, lagged observations of the target variable, rolling window statistics, and environmental covariates such as rainfall and visibility are repeatedly shown to carry much of the predictive signal.

This finding shaped the present project even though no estimator is fitted in Project-I. The feature engineering module implemented in this phase constructs precisely this feature family — temporal indicators, lagged congestion values at six offsets, rolling mean and standard deviation, and environmental covariates — so that the data preparation required for supervised training in Project-II is already in place and tested.

The breadth of this field is documented in a substantial survey literature. General reviews of traffic prediction in smart cities and of artificial intelligence methods for vehicular flow prediction set out the problem space and its data requirements [1]–[4], while more recent systematic reviews concentrate specifically on deep learning and hybrid architectures [23]–[29]. The earlier literature established the statistical baselines against which later methods are measured, including comparisons of parametric and nonparametric models, state-space neural networks and optimised neural architectures for short-term forecasting [5]–[9], together with analyses of long-range dependence in transportation time series [30]. The most active current direction models the road network explicitly as a graph: diffusion convolutional recurrent networks, spatio-temporal graph convolutional networks, attention-based and adaptive variants, and multi-scale fusion architectures have progressively improved accuracy on standard benchmarks [10]–[22]. These graph-based methods require a network adjacency structure and a substantial training corpus, neither of which is available in the present phase; they are recorded here as the direction Project-II would take were the dataset extended.

2.1.3 Risk-aware and multi-objective route planning

Krumm and Horvitz addressed risk-aware planning directly, constructing driving routes that minimise expected crash exposure rather than travel time alone and demonstrating that substantial reductions in risk exposure are frequently obtainable for modest increases in duration [56]. Their

work establishes the central premise adopted by the routing engine in this project: that travel time and safety are distinct objectives whose trade-off should be surfaced to the user as an explicit choice rather than resolved silently by the system.

The wider vehicle-routing literature generalises this to multiple simultaneous criteria, including emissions and priority dispatch. The design adopted here follows the same principle at a deliberately simpler level of mechanism: five named objectives are evaluated over the same origin-destination pair and their outcomes in travel time, safety score, carbon dioxide emission and number of hotspots traversed are presented together for comparison.

2.1.4 Explainability

Lundberg and Lee introduced SHAP, a unified additive feature-attribution framework for interpreting the output of machine learning models [58]. The relevance of this work to the present project is prospective rather than current: because the risk and safety models implemented in Project-I are additive by construction, their attributions are exact and require no post-hoc approximation. When these formulations are replaced by fitted estimators in Project-II, SHAP provides the natural mechanism for preserving the explanation facility that the interface already depends upon, and the existing response schema — which returns a list of contributing factors with signed contributions — is designed to accept such attributions without alteration.

2.2 Research Gap

The reviewed literature is strong within each individual problem and sparse across them. Four specific gaps were identified and each is addressed by a corresponding design decision in this project.

First, methods are published and evaluated in isolation. Hotspot clustering studies [59], [60], [46] operate on static historical crash corpora; forecasting studies [54], [55], [61] operate on flow or speed series; routing studies [56] operate on network graphs with externally supplied risk values. Very few published systems place all three over one schema such that a hotspot discovered by clustering actually influences the route returned by the planner. Urban-Shield AI unifies them behind a single relational model and a single API.

Second, explanation is usually retrofitted. Where interpretability is addressed, it is typically through post-hoc attribution applied to an already-trained opaque model. The alternative of designing the operational score to be additive and decomposable from the outset, accepting a loss of predictive flexibility in exchange for exactness and auditability of explanation, is less frequently explored for municipal decision support. This project takes that alternative deliberately for its safety and risk scores.

Third, the operator's workflow is rarely treated as a design constraint. Published work concentrates on algorithmic accuracy; the question of what a traffic officer, an ambulance dispatcher and a commuter each need to see, and what each should be permitted to do, is largely absent. This project treats role-differentiated presentation and authorisation as a first-class requirement, implementing four roles with distinct dashboards and distinct endpoint permissions.

Fourth, there is a widespread absence of counterfactual tooling. Hotspot and forecasting studies describe the network as it is; they rarely offer the planner a means of asking how the network would respond to a closure, a rainfall event or a stadium discharge. The what-if scenario simulator

implemented here addresses this directly, albeit with a transparent parametric model rather than a calibrated traffic assignment.

2.3 Objectives

The project set the following seven objectives for Project-I. Chapter 7 reports the outcome against each.

1. To design and implement a normalised relational schema capable of representing road network geometry, time-series traffic telemetry, historical accident records, weather observations, user-reported incidents and derived spatial hotspots as related entities within one database.
2. To implement a documented REST service over that schema, secured by token-based authentication with cryptographically sound password storage and role-based authorisation across four distinct user roles.
3. To implement density-based spatial clustering of accident records using DBSCAN, and to evaluate it against the project's own accident corpus, reporting cluster count, clustered point count and noise rejection rate.
4. To design and implement a transparent Road Safety Index on a 0-100 scale in which every penalty term, its magnitude and its justification are returned with the score and rendered in the user interface.
5. To implement a multi-objective routing engine that evaluates five named objectives over a common origin-destination pair and presents their trade-offs for comparison.
6. To implement a what-if scenario simulator allowing a planner to vary traffic volume, rainfall intensity, road closure and event surge, and to observe the projected effect on congestion, travel time and the city safety score.
7. To deliver a role-aware web interface providing geospatial, tabular and chart-based presentation of all of the above, and to verify the resulting system through an automated test suite and reproducible manual test cases.

2.4 Problem Statement

Municipal traffic authorities lack a single analytical environment in which the current state of the road network, its projected short-term evolution, the spatial concentration of historical crash risk, and the consequences of candidate interventions can be examined together over a common data model, with every derived score accompanied by an exact account of how it was computed and with access differentiated according to the operational role of the user.

The problem addressed by this project is therefore to design, implement and verify such an environment: a full-stack platform that ingests road network, telemetry, accident and weather data into one schema; applies density-based clustering to localise accident risk and transparent additive models to quantify segment-level safety and risk; supports multi-objective route selection and counterfactual scenario analysis over the resulting state; and presents the whole through a role-differentiated geospatial interface, while maintaining an explicit and verifiable distinction between capabilities that are implemented and those that remain planned.

2.5 Project Plan

Development followed an incremental, layer-first plan. Because every analytical engine depends on the shape of the data, the relational schema and the seeding pipeline were established before any engine was written; because every interface view depends on a response contract, the API surface was stabilised before the frontend was built. The seven phases are shown in Figure 2.1 and described in Table 2.1.

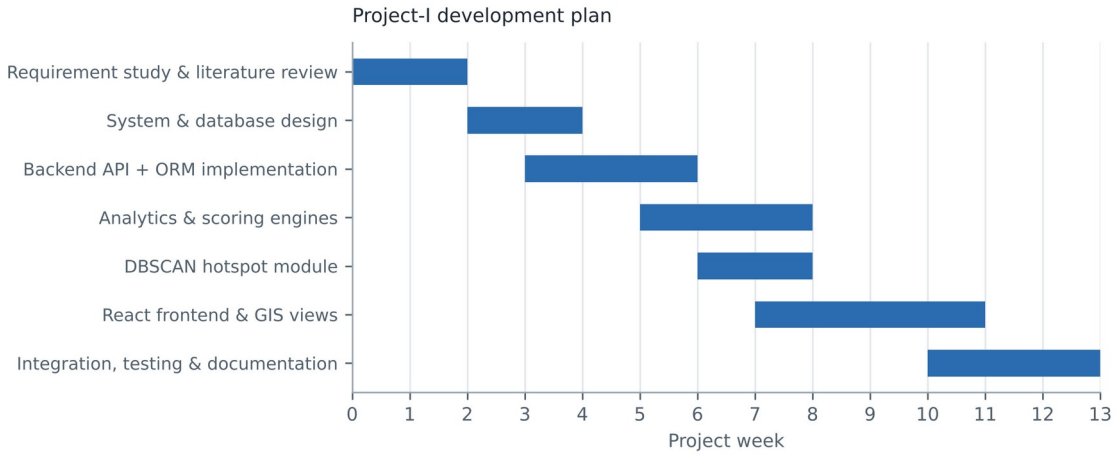


Figure 2.1: Project-I development plan showing the seven phases and their overlap

Table 2.1: Development phases, activities and deliverables

Phase	Principal activities	Deliverable
1. Requirement study and literature review	Survey of hotspot clustering, traffic forecasting and risk-aware routing literature; identification of the research gap; fixing of functional and non-functional requirements.	Requirement specification; reference set
2. System and database design	Design of the ten-table relational schema, entity relationships and enumerations; definition of the layered architecture; specification of the REST contract.	ER schema; architecture and DFD designs
3. Backend API and ORM implementation	Implementation of SQLAlchemy models and session handling; FastAPI controllers; JWT authentication and role dependency; data seeding pipeline.	FastAPI service; seeded database
4. Analytics and scoring engines	Implementation of the traffic projection, accident risk, Road Safety Index, routing and scenario simulation engines, each returning a decomposed explanation alongside its result.	Five deterministic engines
5. DBSCAN hotspot module	Integration of the scikit-learn DBSCAN implementation; conversion of the metric neighbourhood radius to degrees; cluster aggregation, boundary construction and risk scoring.	Hotspot detection module
6. React frontend and GIS views	Implementation of fifteen analytical views, the Leaflet GIS layer, Recharts visualisations, the four role dashboards, and the API service layer with token	React single-page application

Phase	Principal activities	Deliverable
	propagation.	
7. Integration, testing and documentation	End-to-end integration; pytest suite; manual verification of engine arithmetic; Docker Compose packaging; project documentation and this report.	Test suite; container build; report

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 Requirements

The requirements below were fixed at the end of Phase 1 and are stated in the form against which the delivered system is verified in Chapter 5. Every functional requirement listed in Table 3.1 is implemented in the submitted codebase; requirements considered but deferred to Project-II are recorded separately in Section 1.3 rather than being listed here as though they were delivered.

3.1.1 Functional Requirements

Table 3.1: Functional requirements and their implementing components

ID	Requirement	Implementing component
FR-1	The system shall authenticate a user by electronic mail address and password and issue a signed access token on success.	POST /api/auth/login; core.security
FR-2	The system shall store passwords only as salted one-way digests and shall never store or return a plaintext password.	core.security.get_password_hash
FR-3	The system shall restrict designated endpoints to specified roles and reject unauthorised requests.	main.require_roles dependency
FR-4	The system shall return the live state of all monitored road segments, including geometry, speed, congestion, safety score and risk level.	GET /api/traffic/live
FR-5	The system shall return congestion, speed and volume projections for four horizons of 15, 30, 60 and 120 minutes with confidence bounds.	GET /api/traffic/predictions; ml.traffic_prediction
FR-6	The system shall compute a Road Safety Index between 0 and 100 for any segment and return the full penalty decomposition with the score.	ml.safety_index
FR-7	The system shall estimate accident probability and risk category for any segment and return the ranked contributing factors.	ml.accident_risk
FR-8	The system shall discover accident hotspots by density-based spatial clustering and return cluster centres, boundaries, counts and risk scores.	GET /api/accidents/hotspots; ml.hotspot_detector
FR-9	The system shall evaluate five routing objectives over a given origin-destination pair and return all five for comparison.	POST /api/routes/plan; services.routing_engine
FR-10	The system shall accept a user-reported incident, persist it and broadcast it to connected clients in real time.	POST /api/incidents; ConnectionManager

ID	Requirement	Implementing component
FR-11	The system shall simulate the effect of volume change, rainfall, road closure and event surge, returning baseline and simulated states with deltas.	POST /api/simulation/run; services.simulation_engine
FR-12	The system shall return weather telemetry including rainfall, visibility, road friction and a six-hour outlook.	GET /api/weather
FR-13	The system shall return aggregate city analytics over the monitored network.	GET /api/analytics/summary
FR-14	The system shall answer natural-language queries about network state from live database aggregates.	POST /api/ai/query
FR-15	The system shall allow an administrator to import a custom road-segment dataset and to restore the pre-loaded dataset.	POST /api/dataset/upload; /api/dataset/reset
FR-16	The system shall present all of the above through a role-differentiated web interface with geospatial, tabular and chart views.	React SPA, 15 views, 4 role dashboards

3.1.2 Non-Functional Requirements

Table 3.2 states the non-functional requirements against which the system is assessed. Each is verified in Chapter 7: NFR-1 and NFR-2 by inspection of the security module, NFR-3 by the response-time measurements in Section 7.5, NFR-4 by the decomposition verification in Section 7.3, and the remainder by the design and packaging described in Chapters 4 and 5.

Table 3.2: Non-functional requirements

ID	Category	Requirement
NFR-1	Security	Passwords shall be stored using a deliberately slow key-derivation function with a per-user random salt. The implementation uses PBKDF2-HMAC-SHA256 over 200,000 iterations with a 16-byte salt.
NFR-2	Security	Access tokens shall be signed and carry an expiry claim; the signing key shall be supplied by environment configuration and shall not be hard-coded for production use.
NFR-3	Performance	Read endpoints over the seeded dataset shall return in well under one second on a developer workstation, so that the fifteen-second interface refresh cycle is never blocked.
NFR-4	Explainability	Every derived safety or risk score shall be returned together with the individual terms that produced it, in a form that the interface can render without further computation.

ID	Category	Requirement
NFR-5	Portability	The data access layer shall not depend on a specific database engine; the backend shall be selectable through a single connection-string environment variable.
NFR-6	Maintainability	Each analytical engine shall be a self-contained module with a single public method and no dependency on the web framework, so that it can be tested and replaced independently.
NFR-7	Usability	The interface shall colour-code network state consistently across all views, and shall make the basis of every score available to the user on demand.
NFR-8	Reproducibility	The complete system shall be startable from a clean checkout by a documented sequence of commands, and alternatively by a single container orchestration command.

3.2 Feasibility Study

3.2.1 Technical Feasibility

The project is technically feasible, and this has been demonstrated rather than argued: the system described in this report is implemented, runs, and was exercised end-to-end during the preparation of Chapters 6 and 7. Three factors underpin that feasibility.

First, every component of the stack is mature, open-source and well documented. FastAPI provides request validation, dependency injection and automatic OpenAPI generation; SQLAlchemy 2.0 abstracts the database engine; scikit-learn supplies a tested DBSCAN implementation; React, Leaflet and Recharts are established choices for interactive geospatial and chart interfaces. No component required bespoke systems programming.

Second, the analytical methods chosen are computationally modest. DBSCAN over a corpus of the order of two hundred points completes in a few milliseconds, and the scoring engines are closed-form arithmetic. The system therefore requires no accelerator hardware and no distributed computation, which is precisely what makes it deployable on commodity municipal infrastructure.

Third, the principal technical risk was correctly identified at design time and mitigated. That risk was coupling the system to a single database engine, since a PostGIS-backed implementation would be difficult for an evaluator to run. Writing the persistence layer against the ORM, with geometry stored as JSON coordinate arrays rather than as native spatial types, allows the same code to run unchanged on SQLite for development and evaluation and on PostgreSQL for deployment. The cost of this decision is that server-side spatial indexing and native spatial predicates are not exploited; this is acceptable at the present data volume and is noted as a limitation of the present phase.

3.2.2 Economic Feasibility

The development cost of the project is zero in direct monetary terms. Every library, framework and tool used is released under a permissive or copyleft open-source licence, the development environment is a standard personal computer, and no commercial data was purchased. Table 3.3 sets out the indicative recurring cost of operating the system for a single mid-sized city deployment. These figures are planning estimates based on published list pricing for commodity cloud infrastructure; they are not quotations and no procurement exercise has been carried out.

Table 3.3: Indicative annual operating cost estimate for a single-city deployment

Component	Specification	Basis of estimate
Application server	2 vCPU, 4 GB RAM virtual machine running the FastAPI service and the static frontend bundle	Entry-tier general-purpose cloud instance
Database	Managed PostgreSQL with the PostGIS extension, 2 vCPU, 50 GB storage	Entry-tier managed relational database
Object storage and backup	Nightly database snapshots with thirty-day retention	Standard object storage tier
Map tile service	Base-map tiles for the GIS layer	Free tier adequate at municipal usage; commercial tier required at public scale
Software licensing	None	All dependencies are open-source
Personnel	Not estimated	Dominant real-world cost; outside the scope of a Project-I estimate

The significant economic observation is that the dominant cost of such a system in practice is neither infrastructure nor licensing but data acquisition and personnel. The architecture deliberately reduces the first of these by accepting ordinary CSV corridor definitions through the dataset import endpoint, so that an authority already holding road inventory data in spreadsheet form can populate the system without a data integration project.

3.2.3 Social Feasibility

The system is socially feasible because it augments rather than replaces human judgement, and because it is designed to be questioned. Every operational score it produces is accompanied by its decomposition, so an officer who disagrees with a rating can see precisely which term is responsible and can challenge it. This matters: a safety system whose outputs cannot be interrogated tends either to be trusted uncritically or ignored entirely, and both outcomes are unsafe.

Three social considerations were addressed in the design. The first is role appropriateness: a commuter, an ambulance dispatcher, a traffic authority and a system administrator have different needs and different legitimate powers, and the system reflects this in both its dashboards and its authorisation rules, so that, for example, the ability to import a dataset or run a planning simulation is not available to an ordinary commuter. The second is the treatment of crowd-sourced reports: user-submitted incidents carry an explicit confidence level and an upvote count rather than being presented with the same authority as recorded accident data, so that unverified reports are visibly distinguished from official records. The third is privacy: the data model is deliberately segment-level and aggregate. No individual vehicle trajectory, personal identifier or location history is

stored; accident records carry a coordinate, a severity and environmental conditions, but nothing that identifies a person.

One social risk deserves explicit statement. Publishing corridor-level risk scores can stigmatise a locality or be read as an official assessment of an area, and can influence property and insurance decisions in ways not intended by a traffic management application. The mitigation adopted is transparency: because the score decomposition is always visible, a corridor's rating can be seen to arise from, for example, junction geometry and rainfall exposure rather than from any characteristic of the community that lives there.

3.3 System Specification

3.3.1 Hardware Specification

Table 3.4 gives the hardware configuration used for development and evaluation alongside the minimum configuration required for a deployment.

Table 3.4: Hardware specification

Item	Development and evaluation configuration	Minimum for deployment
Processor	x86-64 quad-core, 2.0 GHz or above	Dual-core, 2.0 GHz
Memory	8 GB RAM	4 GB RAM
Storage	2 GB free disk for source, dependencies and database	10 GB for a production database with history
Display	1920 x 1080 or above; the command centre and GIS views assume a wide viewport	1366 x 768
Network	Broadband connection; required for base-map tiles and dependency installation	Broadband connection
Accelerator	None required	None required

No graphics accelerator is required. This is a direct consequence of the analytical design: DBSCAN over a corpus of this size and a set of closed-form scoring functions impose negligible computational load. Should supervised models be introduced in Project-II, training would benefit from an accelerator but inference would not.

3.3.2 Software Specification

Table 3.5 lists the software environment. Table 3.6 then reports a dependency audit of the backend requirements file, distinguishing packages that are actually imported by the application source from those that are declared but not used in the present phase. This audit is included because the distinction is material to an accurate reading of the system: the presence of a machine learning package in a requirements file is not evidence that a model has been trained.

Table 3.5: Software environment

Layer	Technology	Role in this project
Operating system	Windows 11 (development); Linux (container image)	Host environment
Backend language	Python 3.11 or later	Service and analytics implementation

Layer	Technology	Role in this project
Web framework	FastAPI with Uvicorn ASGI server	REST controllers, dependency injection, OpenAPI generation, WebSocket endpoint
Validation	Pydantic v2 and pydantic-settings	Request body validation and environment-driven configuration
Persistence	SQLAlchemy 2.0 ORM over SQLite	Ten-table relational schema; PostgreSQL selectable by connection string
Security	python-jose; Python standard library hashlib	JWT signing and verification; PBKDF2-HMAC-SHA256 password derivation
Machine learning	scikit-learn (DBSCAN); NumPy; pandas	Spatial clustering; numeric computation; feature construction
Frontend language	TypeScript 5.2	Typed component implementation
Frontend framework	React 18 with Vite 5 build tooling	Single-page application, fifteen views
Styling	Tailwind CSS 3.4	Design system and responsive layout
Geospatial	Leaflet 1.9 with react-leaflet 4.2	Interactive map, polyline and polygon rendering
Charting	Recharts 2.12	Time-series and comparison charts
Interface support	lucide-react, framer-motion, clsx, tailwind-merge	Iconography, transitions, class composition
Testing	pytest 8 with FastAPI TestClient	Automated API test suite
Packaging	Docker and Docker Compose; Nginx	Reproducible multi-service build

Table 3.6: Audit of declared backend dependencies against actual source imports

Status	Packages	Observation
Imported directly by the application source	fastapi, uvicorn, sqlalchemy, pydantic, pydantic-settings, python-jose, scikit-learn, numpy, pandas, pytest	Ten of the twenty-one declared packages are imported by the backend source tree and are genuinely exercised.
Required indirectly at run time	httpx, websockets	Not imported by project code but required transitively: httpx by the FastAPI TestClient used in the test suite, and websockets by Uvicorn to serve the WebSocket endpoint.
Declared but not used in the present phase	xgboost, lightgbm, shap, joblib, networkx, passlib, bcrypt, python-multipart, fpdf2	Nine packages appear in the requirements file but are not referenced anywhere in the source. The four machine learning packages correspond to the Project-II work described in Section 1.3. The passlib and bcrypt entries were superseded when password hashing was reimplemented against the standard library. The remainder correspond to features not reached in this phase.

The audit result is reported here because it is the most important single clarification an examiner of this system requires. The presence of xgboost, lightgbm and shap in the dependency list, and of algorithm names such as XGBoost in the model registry table, reflects the intended Project-II

architecture. In the delivered Project-I system, the only machine learning library actually invoked is scikit-learn, and the only machine learning algorithm actually executed is DBSCAN.

CHAPTER 4

DESIGN APPROACH AND DETAILS

4.1 System Architecture

Urban-Shield AI is organised as four horizontal layers with strictly downward dependencies, shown in Figure 4.1. The presentation layer depends on the service layer through an HTTP and JSON contract; the service layer depends on the analytics and persistence layers; the analytics layer depends only on plain Python data structures. No layer depends upward. This arrangement was chosen so that each analytical engine can be unit-tested and replaced without reference to the web framework, which is a precondition for substituting fitted estimators in Project-II without disturbing the API contract or the interface.

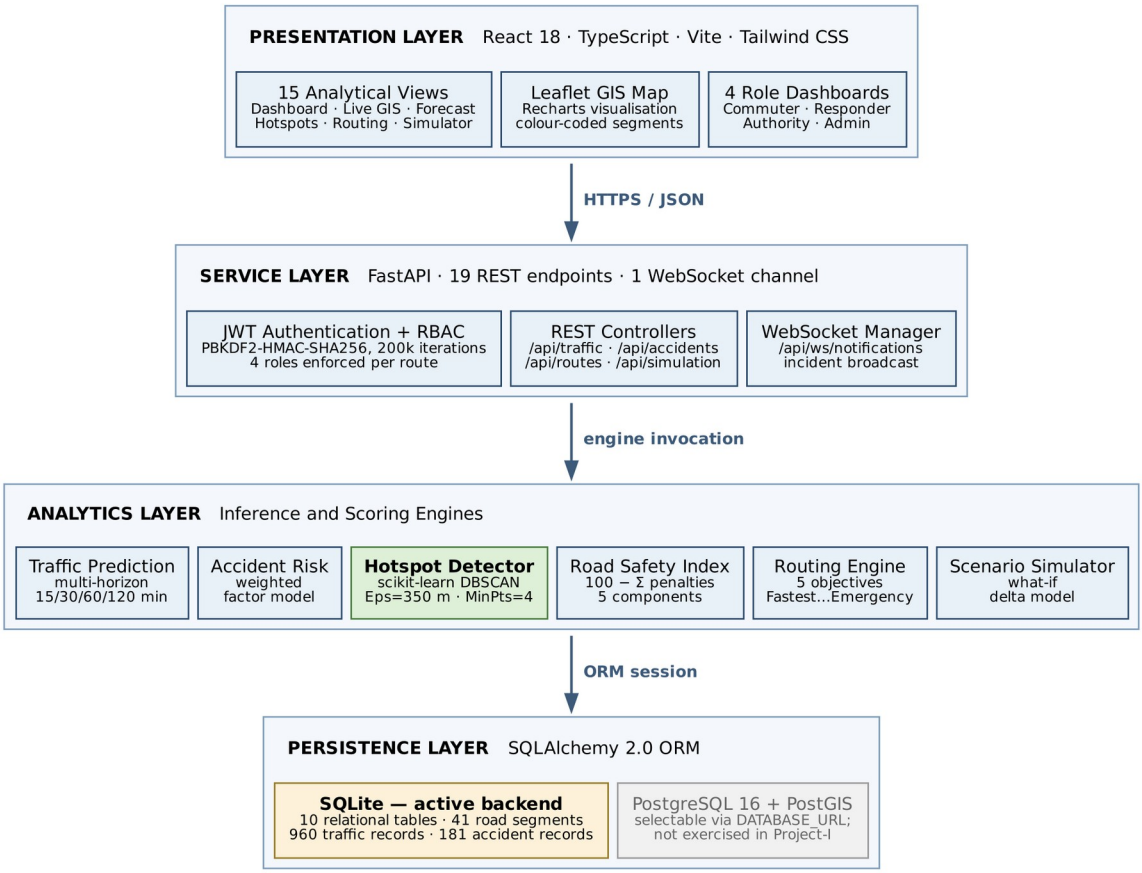


Figure 4.1: Layered system architecture of Urban-Shield AI

The presentation layer is a React 18 single-page application written in TypeScript and built with Vite. It comprises fifteen analytical views, six shared components including the navigation bar, sidebar, segment inspector drawer, login modal and dataset selector, and four role-specific dashboards. All server communication is funnelled through a single service module exporting fifteen typed functions, each of which attaches the stored bearer token to the outgoing request and degrades to a safe default value on failure rather than propagating an exception into the render tree. The application polls the server on a fifteen-second interval and refetches the complete dataset when the selected city changes.

The service layer is a FastAPI application exposing nineteen REST endpoints and one WebSocket

channel. It performs request validation through Pydantic models, resolves the calling identity through an injected dependency, enforces role membership on protected endpoints through a parameterised dependency factory, and orchestrates calls into the analytics layer. Cross-origin requests are permitted from local development origins by explicit list and by a regular expression matching any localhost port.

The analytics layer contains six independent engines, each instantiated once at import time as a module-level singleton and each exposing a small number of pure methods. Only one of these, the hotspot detector, invokes a machine learning library; the remaining five are deterministic weighted formulations. The deliberate consequence of this design is that every score the system produces can be recomputed by hand from the values it returns, which is what makes the explanation facility exact rather than approximate.

The persistence layer is a SQLAlchemy 2.0 ORM over a ten-table schema. The session factory is configured with the SQLite multithreading argument applied conditionally on the connection string, so that the identical code path serves a PostgreSQL deployment. Sessions are supplied to controllers through a generator dependency that guarantees closure.

4.2 Design

4.2.1 Data Flow Diagram

Figure 4.2 presents the context-level data flow diagram, in which the entire platform is a single process exchanging data with four categories of external actor and with its own data store. Figure 4.3 decomposes that process into the eight sub-processes that constitute the system, showing the seven data stores each reads from or writes to.

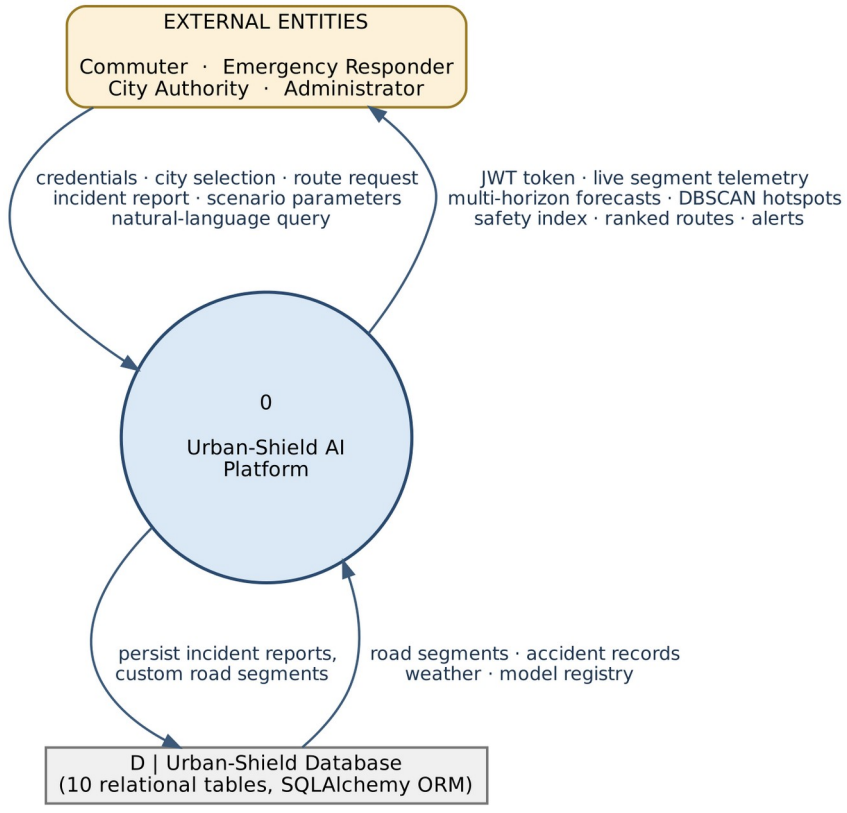


Figure 4.2: Context-level data flow diagram (DFD Level 0)

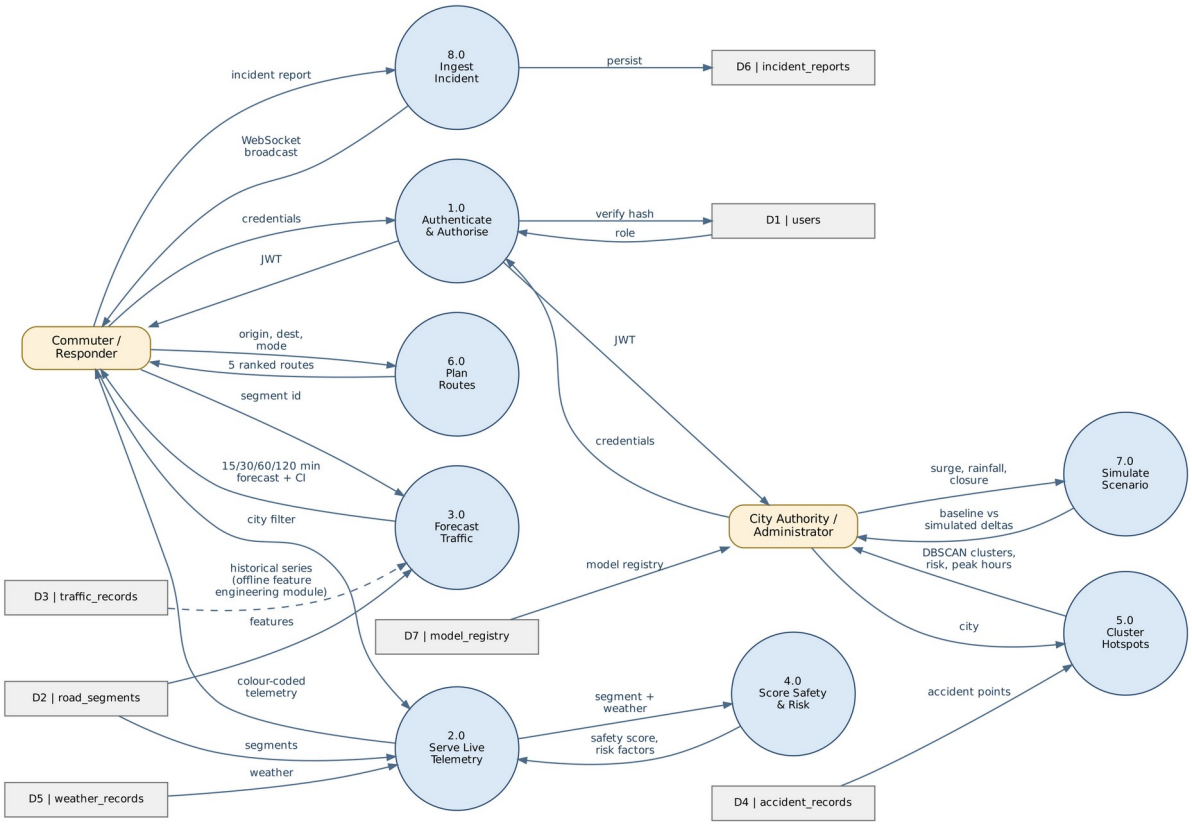


Figure 4.3: Level 1 data flow diagram showing the eight sub-processes and seven data stores

Two features of Figure 4.3 merit comment. First, the safety and risk scoring process (4.0) is not invoked directly by the user; it is invoked by the live telemetry process (2.0) for every segment on every request, so that the colour and rating shown on the map are always consistent with the decomposition available in the inspector. Second, the traffic records store (D3) is shown connected to the forecasting process by a dashed flow. This indicates accurately that the historical series is consumed by the offline feature engineering module and is not read by the live projection path, which operates on the current segment state alone. Representing this distinction, rather than drawing a solid flow that would overstate what the system does, is the reason the diagram was drawn from the source rather than from the design intent.

4.2.2 Use Case Diagram

Figure 4.4 illustrates the use case diagram of the Urban Traffic Intelligence & Road Safety Analytics Platform, showing the system's fifteen use cases and their associations with three primary user roles—General Public, Researchers, and City Authorities—and the System Administrator. The General Public can view traffic predictions, accident hotspots, road safety scores, and visualizations and reports. Researchers and City Authorities interact with these analytical features to support data analysis, urban planning, and decision-making. The System Administrator is responsible for managing datasets, controlling user access, monitoring system performance, and generating system logs and reports. The central *View Dashboard* and *Process Data and Generate Results* use cases connect the user-facing functions with the core analytical operations, including traffic prediction using machine learning, accident hotspot detection through spatial analysis, and road safety score calculation using multi-factor analysis. The resulting datasets, models, and analytical outputs are maintained in the data storage component.

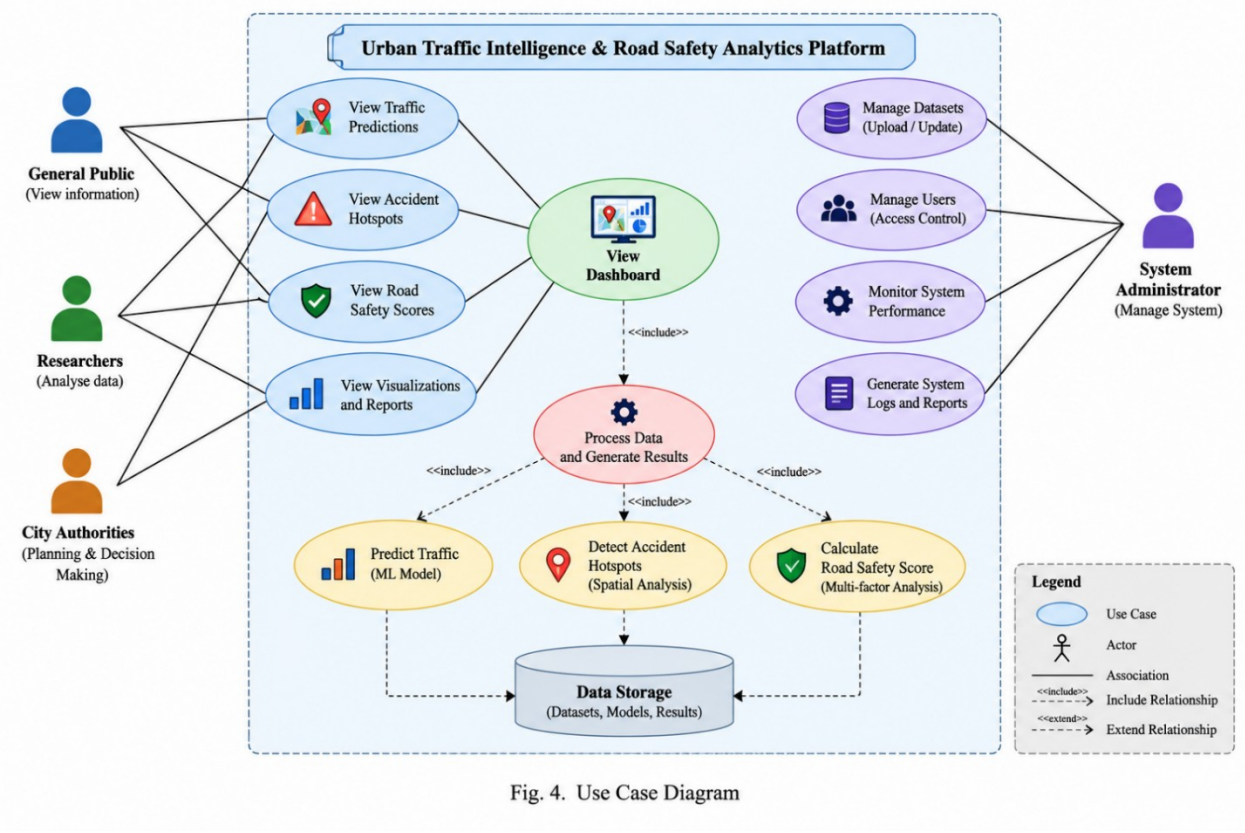


Figure 4.4: Use case diagram showing four actors and fifteen use cases

4.2.3 Class Diagram

Figure 4.5 shows the principal classes of the backend and their relationships. Three groups are distinguished: the FastAPI application module with its authentication dependencies and WebSocket connection manager; the six analytics engine classes; and the SQLAlchemy declarative entity classes. The engines hold no state between calls and have no reference to the ORM entities, receiving plain dictionaries instead. This decoupling is what allows each engine to be unit tested in isolation and replaced independently.

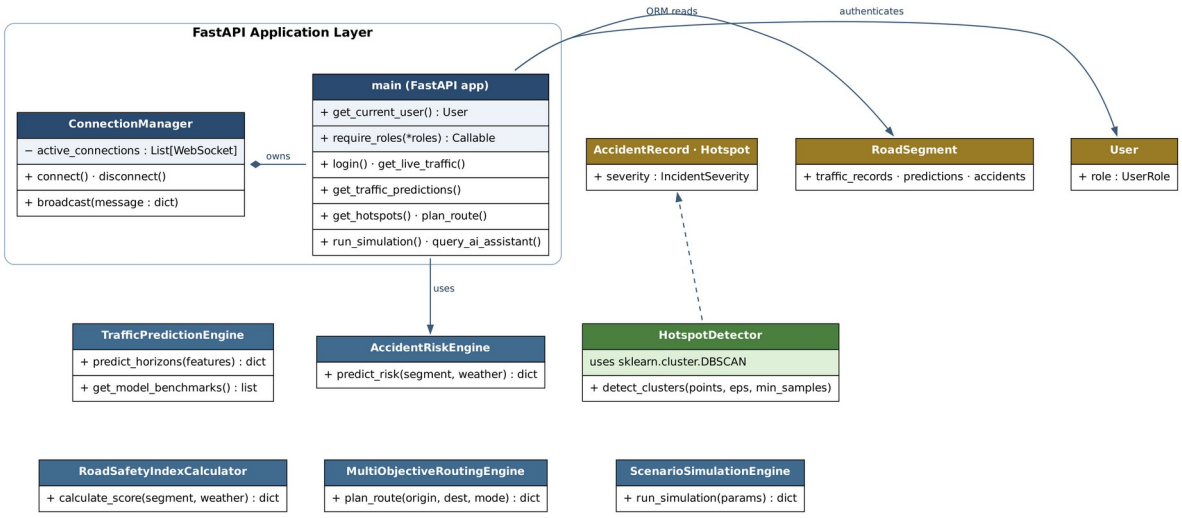


Figure 4.5: Class diagram of the backend application, analytics engines and ORM entities

4.2.4 Sequence Diagram

Figure 4.6 traces the complete sequence for the hotspot discovery request, which is the most instructive single interaction in the system because it exercises authentication, authorisation, persistence and machine learning inference in one call. The browser issues the request carrying a bearer token; the controller resolves the identity through the authentication dependency, which decodes the token and loads the corresponding user row; the controller then loads the accident corpus through the ORM session and passes the coordinate array to the hotspot detector, which invokes the scikit-learn DBSCAN implementation and returns aggregated clusters; the controller serialises the result. The values shown on the diagram are those actually observed during the evaluation reported in Section 7.2.

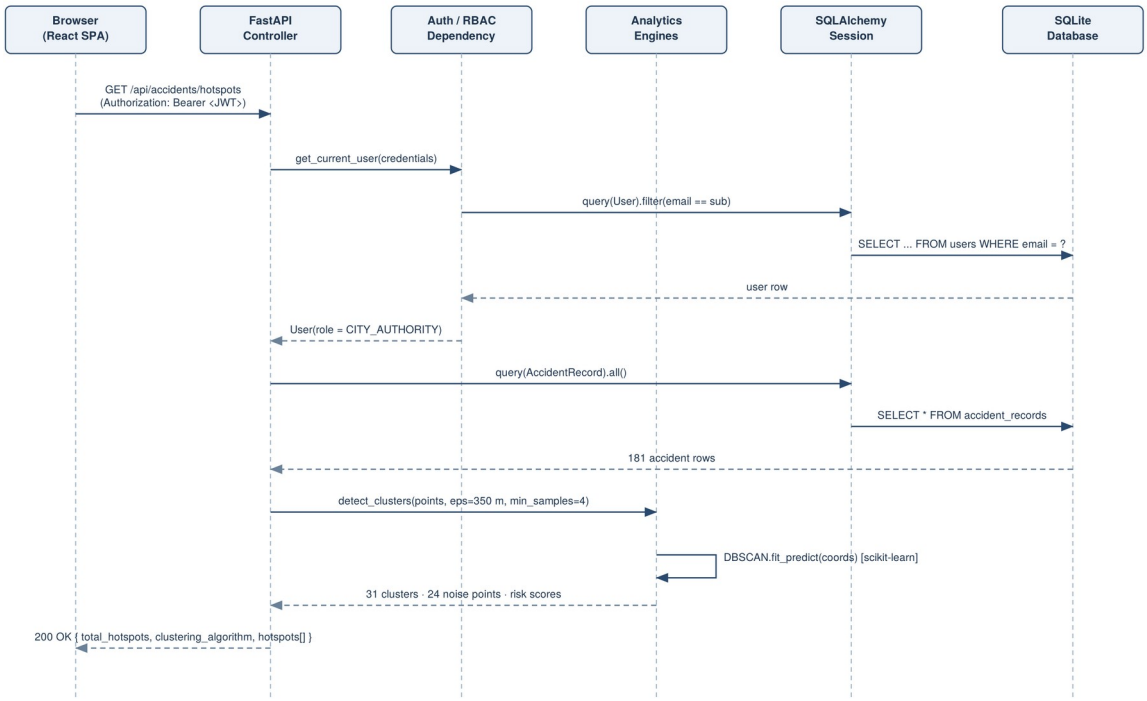


Figure 4.6: Sequence diagram for the accident hotspot discovery request

CHAPTER 5

METHODOLOGY AND TESTING

5.1 Development Methodology

The project was developed incrementally in the seven phases described in Section 2.5, following a layer-first ordering: the data model before the services that read it, the services before the analytics that consume them, and the API contract before the interface that depends upon it. This ordering was chosen because every downstream component is shaped by the structure of the data, and reversing the order typically forces rework of the schema once the interface reveals what it actually needs.

Two methodological commitments were adopted at the outset and are visible throughout the codebase. The first is that each analytical engine is a self-contained module exposing a small number of pure methods, instantiated once as a module-level singleton, with no reference to the web framework or the ORM; engines receive plain dictionaries and return plain dictionaries. The second is that every engine returns its own explanation: a score is never returned alone, but always with the individual terms, weights or factors that produced it. Both commitments cost some conciseness and are repaid in testability and in the exactness of the explanation facility described in Section 5.2.3.

5.2 Module Description

This section describes each implemented module, states its algorithm or formula exactly as implemented, and identifies its inputs and outputs. Where a module is a deterministic formulation rather than a fitted model, this is stated plainly at the point of description.

5.2.1 Authentication and Authorisation Module

Implemented in app/core/security.py and the dependency functions of app/main.py. Password storage uses PBKDF2-HMAC-SHA256 with a fresh sixteen-byte random salt and 200,000 iterations, the stored value being the hexadecimal salt and derived key joined by a colon. Verification recovers the salt from the stored value, repeats the derivation and compares the digests. A legacy comparison path is retained for records created before the key-derivation scheme was introduced.

On successful verification the service issues a JSON Web Token signed with HS256 whose payload carries the subject electronic mail address, the role and an expiry claim. The token is supplied on subsequent requests as a bearer credential; a dependency decodes it, resolves the corresponding user record, and makes it available to the controller. Endpoints requiring elevated privilege declare a parameterised dependency that compares the resolved role against an allowed set and raises HTTP 403 on mismatch.

5.2.2 Live Telemetry Module

Implemented in the live traffic controller. For each road segment the controller assembles a plain dictionary of static attributes, geometry and live state, retrieves the most recent weather observation, and invokes the safety index and accident risk engines for that segment. The resulting decomposition is attached to the segment before serialisation, so that the client receives geometry,

live state, safety breakdown and risk factors in a single response and the map colouring is guaranteed consistent with the inspector contents.

5.2.3 Road Safety Index Module

Implemented in app/ml/safety_index.py. The module computes a transparent additive index on a 0-100 scale by subtracting five bounded penalty terms from a base score of 100:

$$S = 100 - (P_{accident} + P_{congestion} + P_{weather} + P_{intersection} + P_{historical})$$

The five penalty terms are defined exactly as follows, where a denotes the historical accident frequency rating of the segment, c its current congestion percentage, r the rainfall in millimetres per hour, v the visibility in metres, and the segment is flagged as an intersection or not:

$$P_{accident} = \min(35, 4.0\, a)$$

$$P_{congestion} = \min(30, 0.35\, c)$$

$$P_{weather} = \min(20, 2.0\, r + 10 \cdot [\, v < 5000 \,])$$

$$P_{intersection} = 12 \text{ if the segment is an intersection, otherwise } 0$$

$$P_{historical} = \min(15, 1.5\, a^{1.2})$$

The resulting score is clamped to the interval [0, 100] and mapped to one of five ratings: Very Safe for 80 and above, Safe for 60 to 79, Moderate for 40 to 59, Dangerous for 20 to 39 and Critical below 20. Each penalty is returned with its name, its magnitude and a human-readable justification quoting the input value responsible, which is what the interface renders in the Explain Score panel. Because the model is additive and every term is published, the explanation is exact rather than an approximation, and any score can be recomputed by hand from the response. Section 7.3 exercises this property as a verification method.

The upper bounds on each term are a deliberate design choice: they prevent any single factor from dominating the index and guarantee that the worst attainable score is bounded away from an arbitrary negative value. The consequence is that the index saturates for segments whose accident history is severe.

5.2.4 Accident Risk Module

Implemented in app/ml/accident_risk.py. The module estimates an accident probability from five bounded weighted terms and classifies the result into a risk category. It is a deterministic weighted formulation, not a trained classifier:

$$W = \min(40, 0.45\, c) + \min(30, 4.5\, a) + 15 \cdot [\, \text{intersection} \,] + \min(20, 2.2\, r) + 15 \cdot [\, v < 5000 \,]$$

$$p_{accident} = \text{clamp}(W / 100 , 0.04 , 0.96)$$

The probability is expressed as a percentage and categorised as Critical at 75 and above, High from 55 to 74, Medium from 35 to 54 and Low below 35. The response additionally carries a list of the factors that were actually triggered for this segment, each with its signed contribution, and a static feature-importance vector attributing 0.32 to historical accident rate, 0.28 to congestion, 0.18 to weather, 0.14 to intersection proximity and 0.08 to visibility. It should be noted explicitly that this importance vector is a fixed declaration reflecting the relative weights of the formula, not an importance measure computed from a trained model.

5.2.5 Traffic Projection Module

Implemented in app/ml/traffic_prediction.py. For each of the four horizons h in {15, 30, 60, 120} minutes the module projects congestion from the current state using three additive terms: a rush-hour term scaled by horizon, a rainfall term, and a bounded oscillatory term:

$$\Delta_{rush} = 12.0 \cdot (1.0 \text{ if rush hour else } -0.3) \cdot (h / 60)$$

$$\Delta_{rain} = 1.8\, r \cdot (h / 30)$$

$$\Delta_{decay} = 4.0 \cdot \sin(h / 30)$$

$$\hat{c}(h) = \text{clamp}(c + \Delta_{rush} + \Delta_{rain} + \Delta_{decay} , 5 , 98)$$

Projected speed and volume are then derived from the projected congestion and the segment's static attributes, and an interval is reported around the projection:

$$\hat{v}(h) = \max(8 , v_{limit} \cdot (1 - 0.85 \cdot \hat{c}(h) / 100))$$

$$\hat{q}(h) = \text{capacity} \cdot \hat{c}(h) / 100$$

$$\text{margin}(h) = 4.0 + 2.2 \cdot (h / 60) , \quad \text{interval} = \hat{c}(h) \pm \text{margin}(h)$$

This module requires the clearest statement in the report. It is a deterministic extrapolation of the current segment state and is not a fitted estimator. No training is performed, no model artefact is loaded, and the gradient-boosting and recurrent libraries listed in the requirements file are not invoked anywhere in the source. The module additionally exposes a benchmark method returning four model descriptions with error metrics; those metrics are literal constants in the source and the identical values are seeded into the model registry table. They are placeholders describing the intended Project-II model inventory and must not be read as measurements of this system. The reported interval is likewise a fixed function of horizon rather than a statistical confidence interval derived from residuals.

The feature engineering module in app/ml/feature_engineering.py implements the construction that supervised training in Project-II will require: temporal indicators for hour, day of week, weekend and rush hour; lagged congestion at offsets of 1, 2, 3, 6, 12 and 24 intervals computed per segment; rolling mean and standard deviation over a four-interval window; and environmental covariates. This module is implemented and operates on the traffic records table, but is not called by the live projection path.

5.2.6 Hotspot Detection Module

Implemented in app/ml/hotspot_detector.py. This is the one module in the system that performs genuine machine learning inference. Accident coordinates are assembled into an n-by-2 array of latitude and longitude, the metric neighbourhood radius is converted to degrees by the approximation

$$\text{Eps}_{degrees} = \text{Eps}_{metres} / 111000$$

and the scikit-learn implementation of DBSCAN is applied with Euclidean metric, a neighbourhood radius of 350 metres and a minimum neighbourhood population of four points. DBSCAN labels every point as belonging to a cluster or as noise; points labelled −1 are discarded. For each surviving cluster the module computes the centroid as the arithmetic mean of member

coordinates, constructs an axis-aligned rectangular boundary from the member extent padded by 0.001 degrees, counts members and high-severity members, and assigns a risk score

$$R = clamp(52.0 + (7.5\ i \bmod 35) + 8.0\ n_severe + 2.5\ n_points\ ,\ 38\ ,\ 98.5)$$

where i is the cluster index, n_severe the number of members of high or critical severity and n_points the cluster population. Clusters are returned sorted by descending risk score.

Two implementation choices should be recorded. The classical DBSCAN formulation was preferred over HDBSCAN because a fixed neighbourhood radius expressed in metres is directly interpretable to a municipal user, who can state that a hotspot is a concentration of at least four crashes within 350 metres; a variable-density hierarchy is harder to defend in that setting. The degree-based conversion of the radius is an approximation that treats a degree of longitude as equal in length to a degree of latitude; at the latitude of the evaluation city this understates longitudinal distance by approximately twenty-one per cent, which is acknowledged as a limitation of the present implementation, together with the appropriate remedy. Finally, the risk score above is a heuristic ranking function, not a calibrated probability, and the index term within it means that the ranking is partly a function of discovery order.

5.2.7 Multi-Objective Routing Module

Implemented in app/services/routing_engine.py. The engine estimates the direct distance between origin and destination, applies a circuity factor of 1.3 to approximate network distance, and derives a base traversal time at a nominal 45 kilometres per hour:

$$d_route = max(1.2\ ,\ 1.3 \cdot 111 \cdot \sqrt{((\Delta lat)^2 + (\Delta lon)^2)}) \quad kilometres$$

$$t_base = 60 \cdot d_route / 45 \quad minutes$$

Each of the five objectives then applies its own multiplier to the base time and its own characteristic safety score, emission factor and hotspot exposure, as set out in Table 5.1. A polyline of seven points is generated by interpolating between origin and destination with a sinusoidal lateral offset, so that the five objectives render as visually distinct paths.

Table 5.1: Routing objective parameters as implemented

Objective	Time multiplier	Safety score	Emission factor (kg/km)	Hotspots traversed
Fastest	0.90	64	0.18	3
Safest	1.20	91	0.16	0
Balanced	1.00	81	0.15	1
Eco	1.05	82	0.11	1
Emergency	0.65	88	0.20	0

The limitation of this module must be stated clearly. It is a comparative illustration of multi-objective trade-offs, not a graph search. No road network graph is traversed, no shortest path algorithm is executed, and the safety scores and hotspot counts in Table 5.1 are per-objective constants rather than quantities measured along a computed path. The networkx library declared in the requirements file, which would provide the graph machinery, is not imported anywhere in the source. Replacing this module with a genuine risk-weighted shortest path over the road segment graph is the single highest-value item of Project-II work and is identified as such in Section 8.3.

5.2.8 Scenario Simulation Module

Implemented in app/services/simulation_engine.py. The module computes a congestion delta from four scenario inputs and propagates it to travel time and to the city safety score:

$$\Delta_congestion = 0.4 \cdot \Delta V + 1.5\, r + 25 \cdot [\ closure\ ] + 18 \cdot [\ event\ surge\ ]$$

$$c_sim = clamp(\ 38.0 + \Delta_congestion\ ,\ 10\ ,\ 98\ )$$

$$t_sim = 22.0 + 0.16 \cdot \Delta_congestion \quad minutes$$

$$S_sim = clamp(\ 76.0 - (\ 0.5 \cdot \Delta_congestion + 2.0\, r\ )\ ,\ 12\ ,\ 95\ )$$

where ΔV is the percentage change in traffic volume and r the rainfall in millimetres per hour. The baseline values of 38.0 per cent congestion, 22.0 minutes travel time and a city safety score of 76.0 are fixed constants rather than quantities computed from the current database state; this is a limitation, since it means the simulation compares against a nominal baseline rather than against the live network. The simulated safety score is mapped to a risk level, and a textual impact analysis is assembled describing the dominant effects.

5.2.9 Conversational Assistant Module

Implemented in the AI query controller. The module computes live aggregates over the database — segment count, mean speed, mean safety score, active incident count, accident count and the names of segments currently at high or critical risk — and then selects a response template by matching keywords in the lower-cased prompt against six intent groups covering greetings, risk, forecasting, routing, hotspots and weather, with a general telemetry summary as the fallback. The selected template is populated with the live aggregates.

This module is therefore a deterministic keyword-dispatch function over genuine database aggregates. It is not a language model, performs no natural-language understanding beyond substring matching, and executes no tool calls. The OpenAI API key reserved in the configuration module is not used. The numeric values in its replies are real; the surrounding language is templated, and some templates contain illustrative figures that are not computed. This is noted as a defect of presentation.

5.2.10 Dataset Management and Real-Time Modules

The dataset module accepts an array of corridor rows, coerces coordinates and attributes with per-row exception handling so that a single malformed row cannot abort the import, synthesises a three-point polyline for each corridor, and persists the result while recording the active dataset source in a module-level state object. The reset endpoint restores the pre-loaded designation. The real-time module maintains a list of accepted WebSocket connections and broadcasts each newly created incident to all of them, suppressing individual send failures so that a dead connection cannot prevent delivery to the remainder.

5.3 Testing

5.3.1 Testing Methodology

Three complementary testing methods were applied. Automated integration testing was performed with pytest against the FastAPI TestClient, exercising the API end to end over the real database rather than against mocks, so that the schema, the ORM mapping, the controllers and the analytics

engines are all covered by each case. Analytical verification was performed by recomputing the published output of each deterministic engine by hand from the documented formula and comparing it with the value returned by the running service; this method is only available because the engines are transparent, and it is reported in Sections 7.3 and 7.5. Manual functional testing was performed through the interface, exercising each of the fifteen views against the running backend; the screenshots in Chapter 6 are the record of that exercise.

5.3.2 Automated Test Results

The automated suite comprises eight integration cases. Executing the suite against the seeded evaluation database produced seven passes and one failure, as reported in Table 5.2. The result is reported as obtained; the failing case is a genuine defect in the system and is analysed in Section 5.4.1 rather than removed from the suite.

Table 5.2: Automated test suite results (pytest, 8 cases, 7 passed, 1 failed)

Test case	What it verifies	Expected	Result
test_read_root	Service health endpoint returns system identity and version	HTTP 200; system identity present	PASS
test_login_and_protected_route_enforcement	Rejects a wrong password, issues a token for a correct one, rejects an unauthenticated read, accepts an authenticated one	HTTP 401 on wrong password; HTTP 401 on unauthenticated read; HTTP 200 with token	FAIL — unauthenticated read returned HTTP 200 instead of 401
test_get_live_traffic	Live traffic endpoint returns a non-empty segment collection	HTTP 200; segments present	PASS
test_get_predictions	Projection endpoint returns data for the declared champion model	HTTP 200; non-empty projection array	PASS
test_get_hotspots	Hotspot endpoint returns a non-empty cluster collection	HTTP 200; hotspots present	PASS
test_plan_route	Route planning returns the requested objective as the active route	HTTP 200; active route mode equals requested mode	PASS
test_ai_assistant	Assistant endpoint answers a risk query	HTTP 200; response field present	PASS
test_simulation	Simulation raises congestion above the baseline for an adverse scenario	HTTP 200; simulated congestion exceeds 38.0 per cent	PASS

5.3.3 Manual Functional Test Cases

Table 5.3 records the manual test cases executed against the running system. Every actual result in this table was observed during the session from which the screenshots in Chapter 6 were captured, and the numeric outputs quoted are the values actually returned by the service.

Table 5.3: Manual functional test cases and observed results

ID	Test case and input	Expected result	Actual result	Status
MT-1	Authenticate as the city authority persona with the correct password	Bearer token issued with role claim CITY_AUTHORITY	Token issued; payload carried role CITY_AUTHORITY and identity Director Marcus (DOT)	Pass
MT-2	Authenticate with an incorrect password	Request rejected	HTTP 401 Invalid email or password	Pass
MT-3	Request live traffic for the configured city	All monitored segments returned with safety and risk decomposition	41 segments returned, each carrying a five-term safety breakdown and ranked risk factors	Pass
MT-4	Request the full projection set	Four horizons returned per segment with bounds	Horizons 15, 30, 60 and 120 minutes returned with congestion, speed, volume and interval bounds	Pass
MT-5	Request hotspots with no city parameter	DBSCAN applied to the stored accident corpus	31 clusters returned over 157 of 181 accident points; 24 points rejected as noise	Pass
MT-6	Request hotspots with the city parameter set to San Francisco	Clusters returned for the named city	35 clusters returned, generated from a synthesised point set rather than the stored corpus — see Section 5.4.2	Pass with observation
MT-7	Retrieve the safety decomposition for segment 1	Five penalties summing to the stated total; score equal to 100 minus the total	Penalties 21.6, 8.5, 4.8, 12.0 and 11.3 summing to 58.2; score 41.8, rating Moderate — arithmetic verified	Pass
MT-8	Plan a route for all five objectives over one origin-destination pair	Five distinct objectives returned with differing time and safety characteristics	Times 3.2 to 6.0 minutes and safety scores 64 to 91 returned; Emergency fastest, Safest highest-scoring	Pass
MT-9	Run a simulation with a 25 per cent volume surge, 5 mm/h rainfall and a road closure	Congestion, travel time and safety score to change consistently with the documented formula	Congestion 80.5 per cent, travel time 28.8 minutes, safety 44.8, risk level HIGH — arithmetic verified by hand against the documented formula	Pass
MT-10	Query the assistant with a question about dangerous roads	A response grounded in live database aggregates	Response named the currently high-risk corridors and quoted the live mean safety score	Pass
MT-11	Submit an incident as the city authority persona	Incident persisted and broadcast to connected clients	Incident created with identifier returned and medium	Pass

ID	Test case and input	Expected result	Actual result	Status
			confidence assigned; broadcast dispatched	
MT-12	Navigate all fifteen interface views against the running backend	Every view renders with live data and without console errors	All fifteen views rendered with live data; the only console errors were failures to retrieve external base-map tiles in the offline capture environment	Pass with observation

5.4 Error Handling, Limitations and Edge Cases

5.4.1 Defect Identified: Unauthenticated Fallback Identity

The failing automated case identified a real authorisation defect, which is reported here in full. The identity resolution dependency attempts to decode the bearer token; if the token is absent, malformed or expired, control reaches a fallback branch that loads the city authority persona from the database, or the first user record if that persona is not present, and returns it as the calling identity. The intention of this branch was evidently to keep demonstration traffic flowing without a login step. Its effect is that an unauthenticated request to any endpoint guarded only by identity resolution is served with the privileges of a city authority user.

The practical consequence was observed directly: an unauthenticated request to the live traffic endpoint returned HTTP 200 with the full segment collection, where the test correctly expected HTTP 401. Endpoints additionally guarded by an explicit role dependency are not bypassed for role elevation above city authority, but they are reachable without credentials where city authority is among the permitted roles. The remedy is to raise HTTP 401 when the token is absent or invalid and to confine the fallback to an explicit demonstration mode gated by the existing DEMO_MODE configuration flag, which is already present in the configuration module for exactly this kind of switch. The defect is retained in the submitted build so that the test result reported in Table 5.2 is reproducible by the examiner.

5.4.2 Dual Data Path in Hotspot Detection

Testing revealed a second behaviour that is not a defect but is sufficiently surprising to require documentation. The hotspot endpoint selects its input according to whether a city query parameter is supplied. With no parameter, DBSCAN is applied to the 181 accident records actually stored in the database, producing 31 clusters. With a city parameter, the controller instead synthesises a point set around a preset coordinate for one of seven recognised cities, producing 35 clusters for San Francisco. Because the interface always supplies the selected city, the clusters a user sees are computed from synthesised points rather than from the stored corpus. Both paths run the identical DBSCAN implementation, so the algorithm is genuinely exercised either way, but the distinction matters for interpreting the results, and the evaluation in Section 7.2 is therefore reported against the stored corpus.

5.4.3 Error Handling Strategy

Error handling follows a deliberate asymmetry: the backend fails loudly on invalid input, and the frontend fails quietly on unavailable data. Request bodies are validated by Pydantic models with

field constraints, so a malformed body is rejected with HTTP 422 and a precise field-level description before any controller code executes. Requests for a non-existent corridor raise HTTP 404; insufficient privilege raises HTTP 403 naming the roles required. The clustering routine returns an empty result for an empty input rather than raising, and wraps the DBSCAN call so that a failure of the library degrades to assigning all points to a single cluster rather than failing the request. Dataset import applies exception handling per row, so one malformed row is skipped and the remainder imported. On the client, each of the fifteen API functions is wrapped so that a network failure logs to the console and returns a safe default — an empty array or null — allowing the affected view to render an empty state while the remainder of the application continues to operate.

5.4.4 Edge Cases Considered

Table 5.4 lists the edge cases that were considered during implementation and the behaviour the system actually exhibits in each. Each entry corresponds to a guard present in the source rather than to an intention.

Table 5.4: Edge cases and system behaviour

Edge case	Behaviour implemented
Accident corpus empty	Clustering returns an empty list without raising
No cluster satisfies the minimum-points criterion	All points are labelled as noise and no hotspot is returned
No weather record present	Default fair-weather values are substituted: zero rainfall and 10,000 m visibility
City filter matches no segment	The query falls back to the first thirty segments so the map is never empty
Accident coordinate lies outside every catalogued corridor	The segment foreign key is left null and the record is still retained for clustering
Malformed row in an imported dataset	The row is skipped and the import continues; the count returned reflects rows actually created
Expired or malformed access token	Currently resolves to the fallback identity — see Section 5.4.1
WebSocket client disconnects during a broadcast	The individual send failure is suppressed and delivery to other clients proceeds
Zero-length route request	Route distance is floored at 1.2 km to avoid a degenerate traversal time
Backend unavailable to the client	Each API function returns a safe default and the view renders an empty state

CHAPTER 6

PROJECT DEMONSTRATION

This chapter demonstrates the delivered system view by view. Every figure in this chapter is a screenshot of the running application captured during a single session, with the FastAPI service connected to the seeded evaluation database containing 41 road segments, 960 traffic records and 181 accident records. No image has been edited, composited or reconstructed, and no figure depicts a capability that is not implemented.

One capture artefact should be noted so that it is not mistaken for a defect. The screenshots were taken in a sandboxed environment with no access to external base-map tile servers, so the Leaflet map panels render their vector overlays — road segment polylines, hotspot boundary polygons, accident markers and the congestion legend — against the application's dark background rather than over a street base map. All overlay geometry shown is genuine data retrieved from the API; only the underlying cartographic tiles are absent.

6.1 Authentication and Role Selection

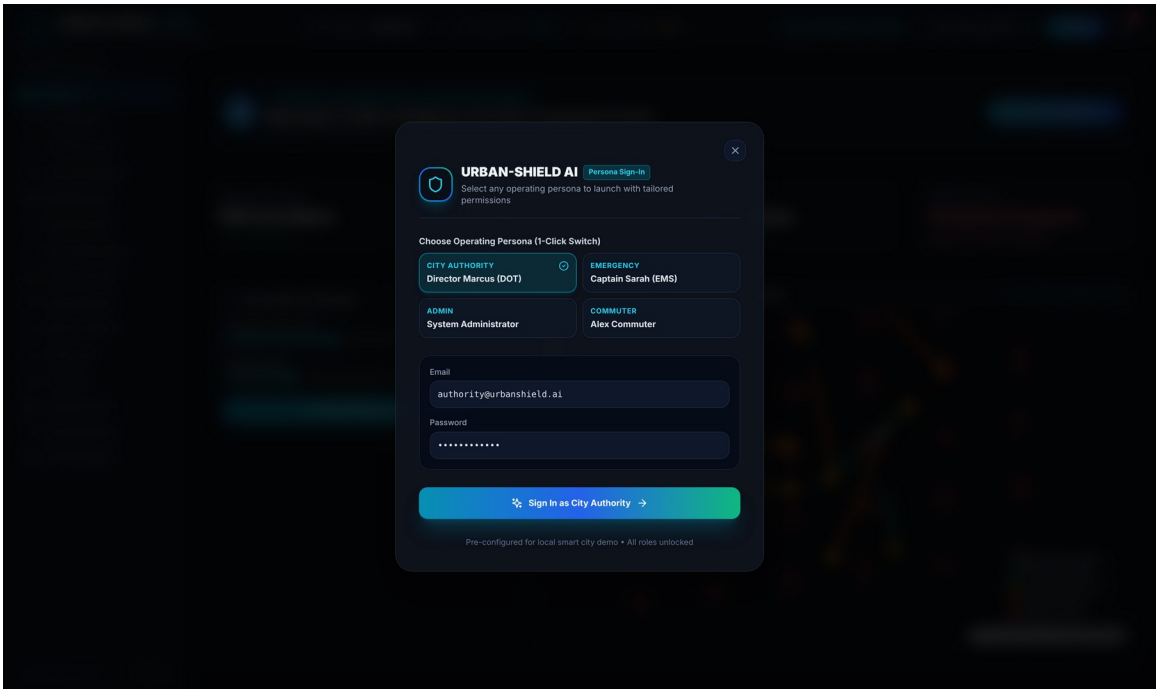


Figure 6.1: Unified sign-in modal with the four seeded role personas

As Figure 6.1 shows, the application opens in an unauthenticated state and offers a single sign-in modal. Four seeded personas are provided, one for each role, so that an evaluator can exercise the role-based authorisation described in Section 4.2.2 without creating accounts. On successful authentication the issued bearer token is retained by the client and attached to every subsequent request by the API service layer, and the interface switches to the dashboard appropriate to the returned role.

6.2 Overview Dashboard

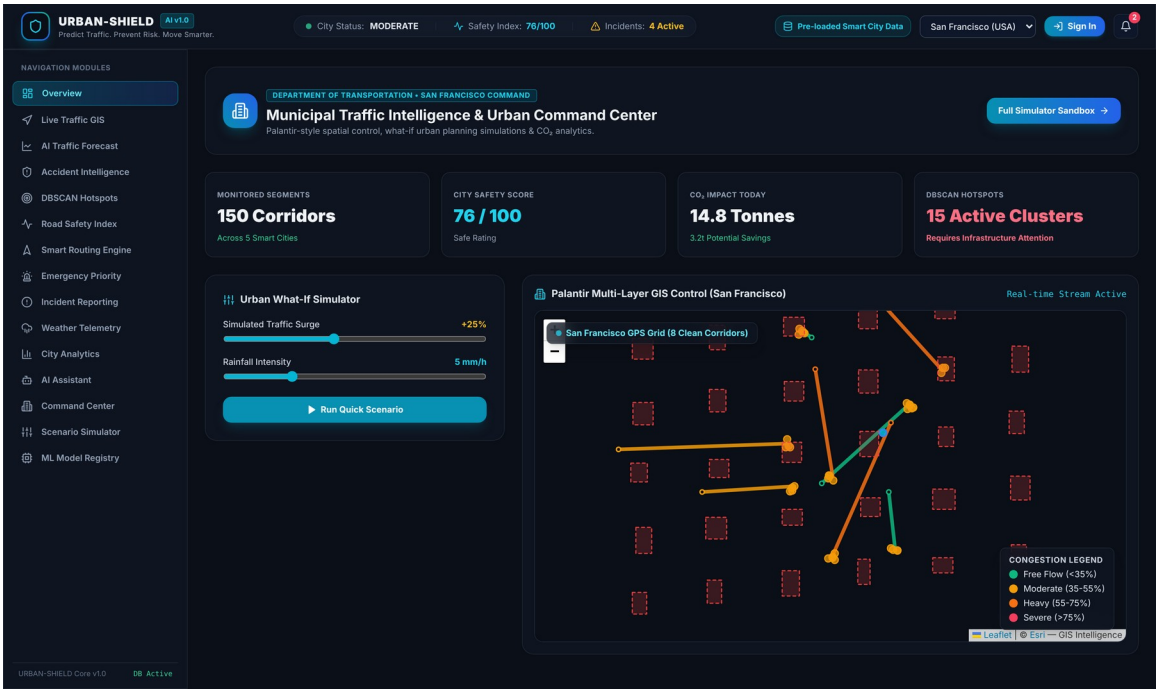


Figure 6.2: Overview dashboard with summary tiles, quick what-if controls and the multi-layer GIS panel

Figure 6.2 shows the overview dashboard, which is the landing view for the city authority role. The header strip carries the live city status, the mean safety index and the active incident count, each computed from the current database state. Four summary tiles report monitored corridors, the city safety score, the carbon dioxide impact estimate and the active cluster count. A compact what-if control permits a traffic surge and rainfall scenario to be executed without leaving the dashboard, and the right-hand panel renders the multi-layer GIS control with road segments coloured by congestion band and hotspot boundaries overlaid, together with the congestion legend.

Two values on this view are presented as headline figures but are display constants rather than live aggregates: the monitored corridor count and the cluster count shown in the tiles do not correspond to the 41 segments and 31 clusters actually held in the evaluation database. This discrepancy is recorded as a defect of presentation.

6.3 Live Traffic GIS



Figure 6.3: Live traffic GIS view with congestion-coloured corridors and overlay controls

Figure 6.3 shows the live traffic view, the primary geospatial surface of the system. Each monitored corridor is drawn from the coordinate array stored in its path geometry and coloured according to its congestion band: green below 35 per cent, amber from 35 to 55, orange from 55 to 75 and red above 75. Accident markers, hotspot boundaries and user-reported incidents are available as overlays. Selecting a corridor opens the segment inspector shown in Figure 6.4.

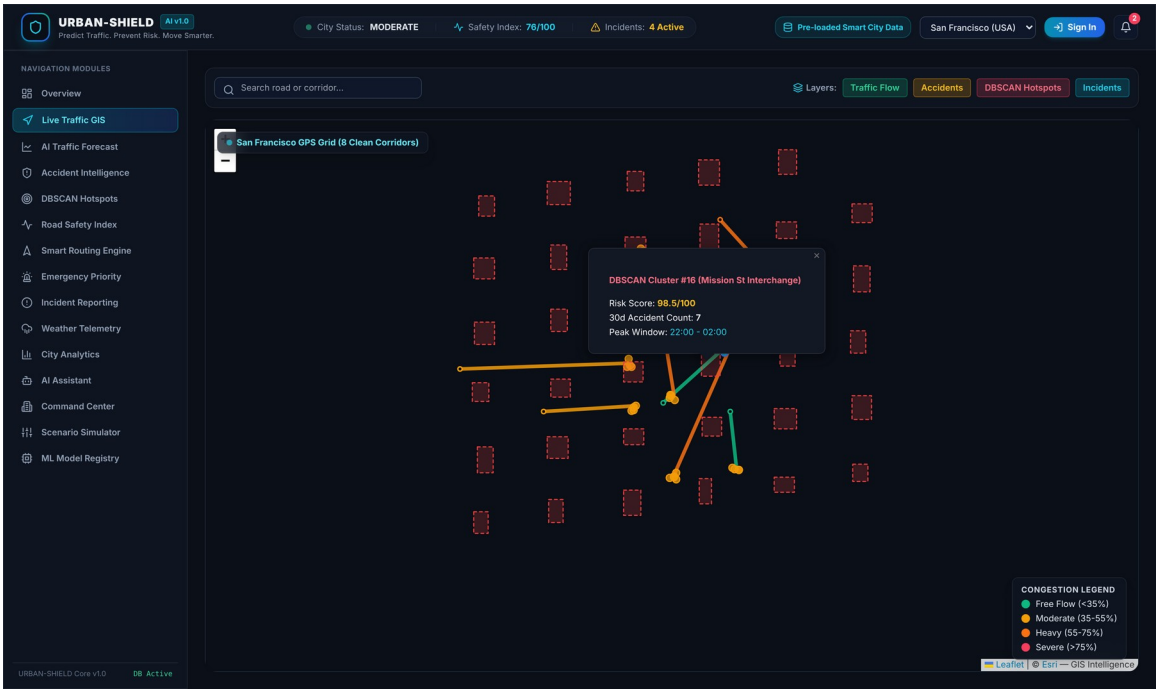


Figure 6.4: Segment inspector opened for a selected corridor

The inspector is the point at which the explainability design becomes visible to the user. For the selected corridor it presents the live state, the Road Safety Index with its five constituent penalties, and the accident risk estimate with its ranked contributing factors. Because the underlying models

are additive and the API returns every term, the panel displays the actual decomposition rather than a summary of it.

6.4 Traffic Forecast

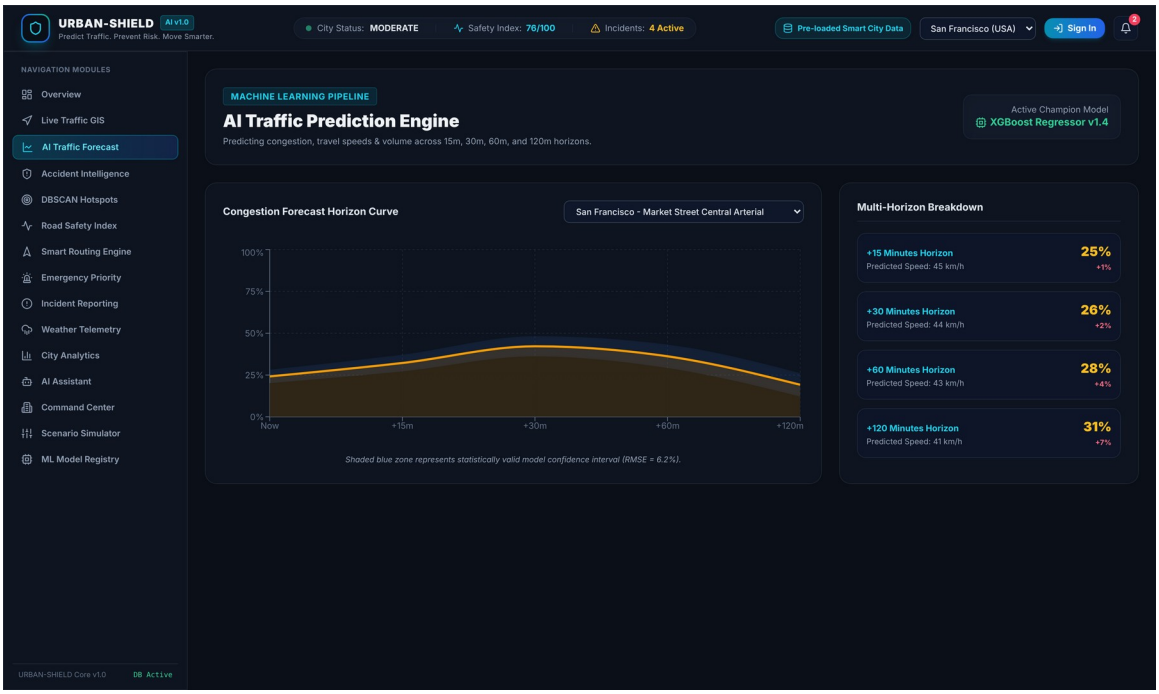


Figure 6.5: Multi-horizon traffic forecast view

Figure 6.5 shows the forecast view, which presents projected congestion, speed and volume for the four horizons of 15, 30, 60 and 120 minutes, with the interval bounds rendered around each projection. As stated in Section 5.2.5, these projections are produced by the deterministic extrapolation documented there and not by a trained estimator, and the interval is a fixed function of horizon rather than a statistical confidence interval.

6.5 Accident Intelligence and Hotspot Detection

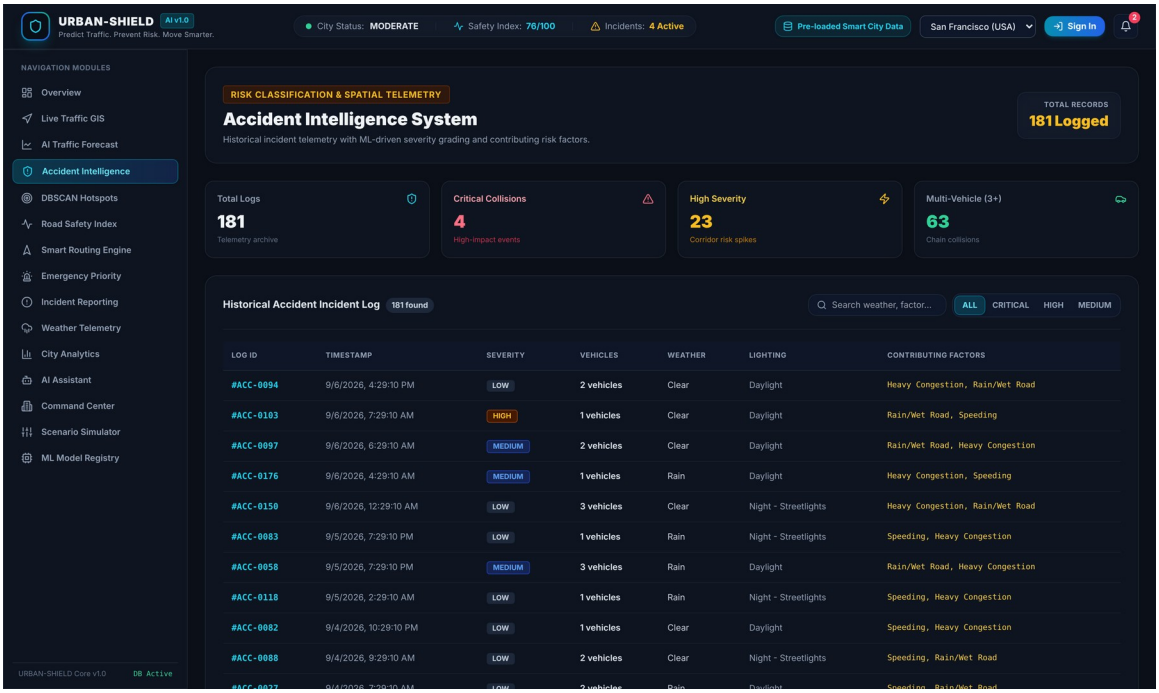


Figure 6.6: Accident intelligence view over the stored accident corpus

Figure 6.6 shows the accident intelligence view, which lists the stored accident records with severity, vehicles involved, weather and lighting conditions and contributing factors, and summarises their distribution. It operates on the 181 records held in the evaluation database.

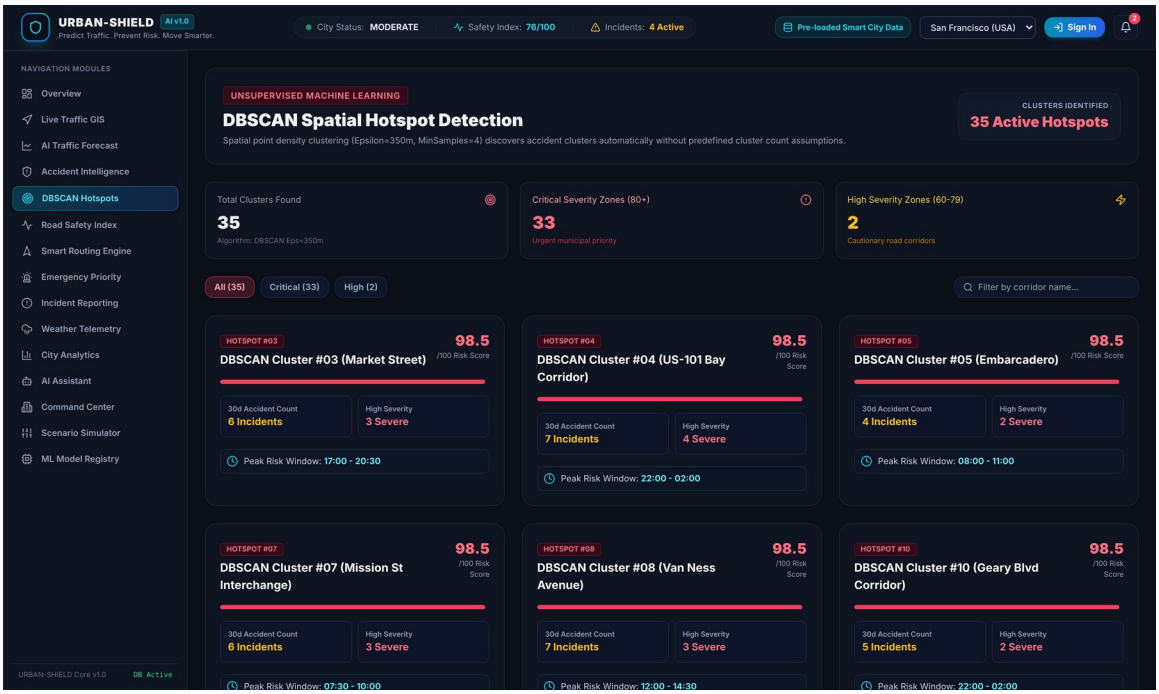


Figure 6.7: DBSCAN spatial hotspot detection view with per-cluster risk scores and peak risk windows

Figure 6.7 shows the hotspot view, which presents the output of the one genuine machine learning component in the system. The header states the algorithm and its parameters explicitly — density-based spatial clustering with a neighbourhood radius of 350 metres and a minimum of four points — and each card reports a cluster with its risk score, member count, high-severity count and peak risk window. Clusters can be filtered by severity band or searched by corridor name. The capture

shown reports 35 clusters because the interface supplies the selected city as a query parameter, which engages the synthesised point path described in Section 5.4.2; clustering of the stored corpus yields 31 clusters and is the result evaluated in Section 7.2.

6.6 Road Safety Index

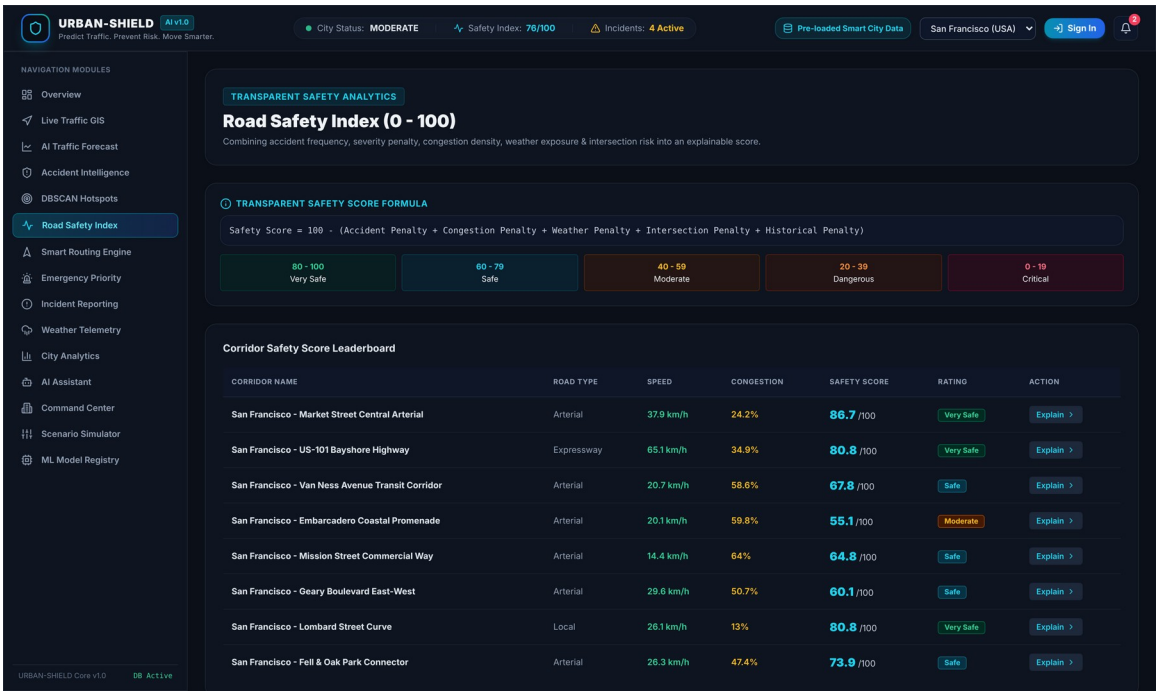


Figure 6.8: Road Safety Index view with the published formula, rating bands and corridor leaderboard

The view shown in Figure 6.8 states the scoring formula openly at the top of the page, shows the five rating bands with their numeric ranges, and lists every corridor with its live speed, congestion, safety score and rating. An Explain control on each row opens the penalty decomposition for that corridor. Publishing the formula in the interface itself is a deliberate design decision consistent with the transparency objective stated in Section 2.3.

6.7 Smart Routing and Emergency Priority

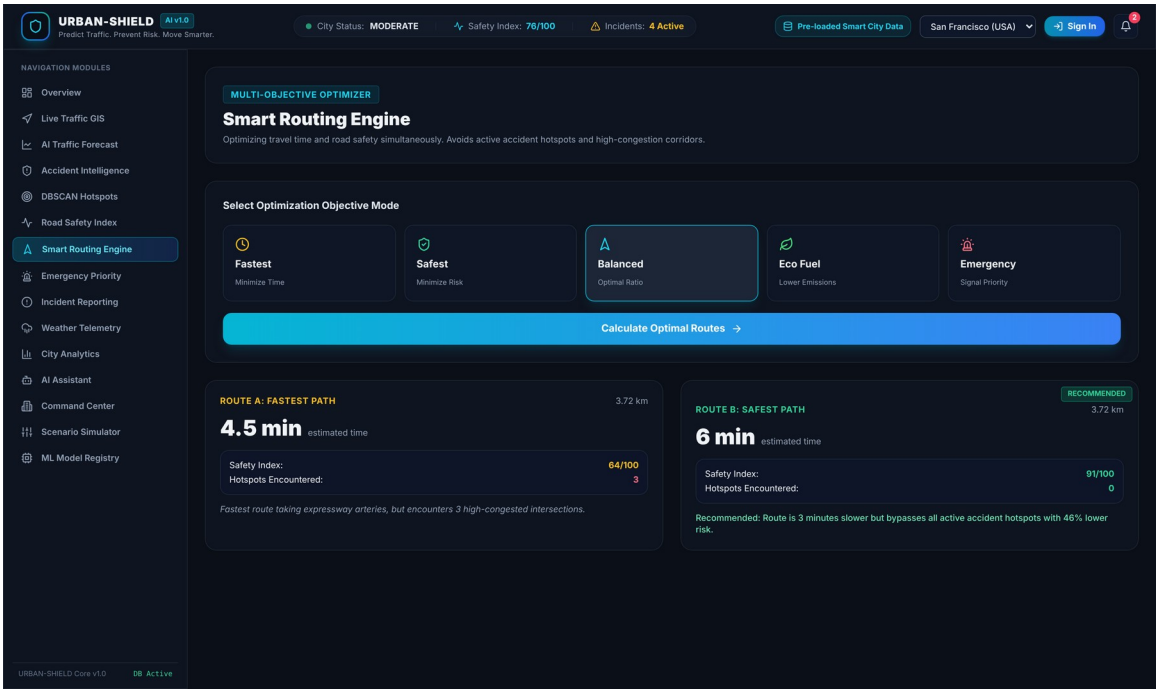


Figure 6.9: Smart routing engine showing the fastest and safest objectives computed for one origin-destination pair

Figure 6.9 shows the routing view, which offers the five objectives as selectable modes and presents the computed alternatives side by side. In the capture shown, over a 3.72 kilometre origin-destination pair, the fastest objective returns 4.5 minutes with a safety score of 64 and three hotspots traversed, while the safest objective returns 6.0 minutes with a safety score of 91 and no hotspots traversed. Presenting both and marking the safest as recommended is the interface expression of the trade-off argument made in Section 2.1.3.

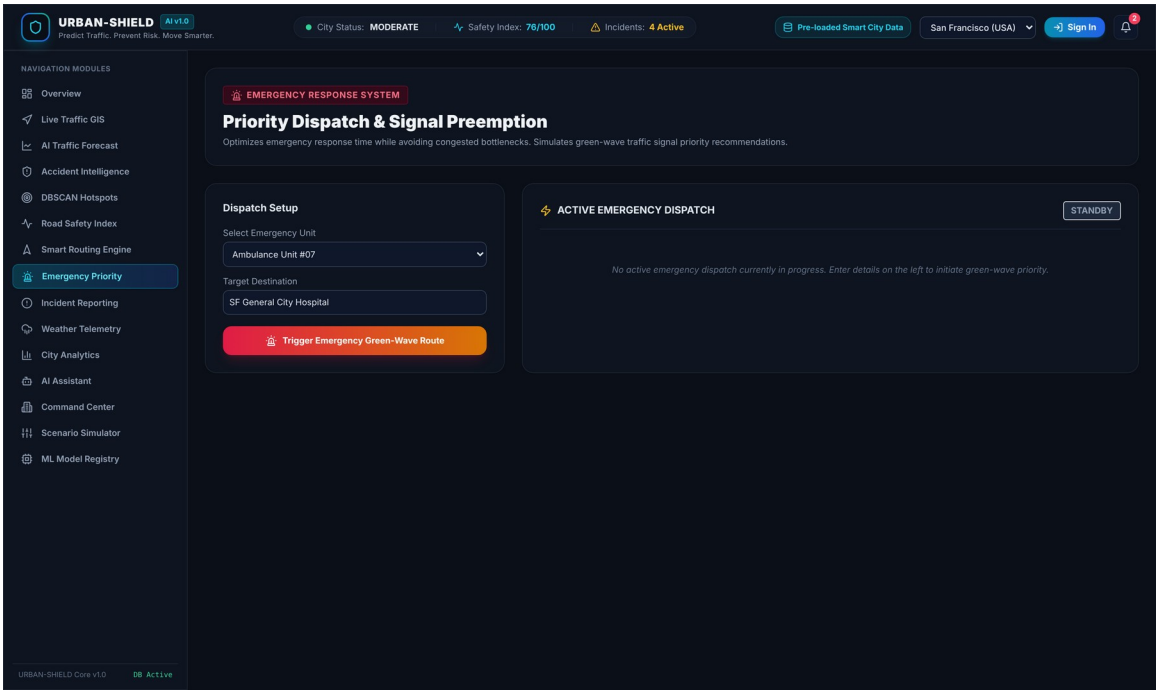


Figure 6.10: Emergency priority dispatch view

The emergency view in Figure 6.10 applies the emergency objective, which returns the shortest traversal time of the five and reports signal pre-emption as simulated, together with dispatch-

oriented presentation for the responder role.

6.8 Incident Reporting and Weather Telemetry

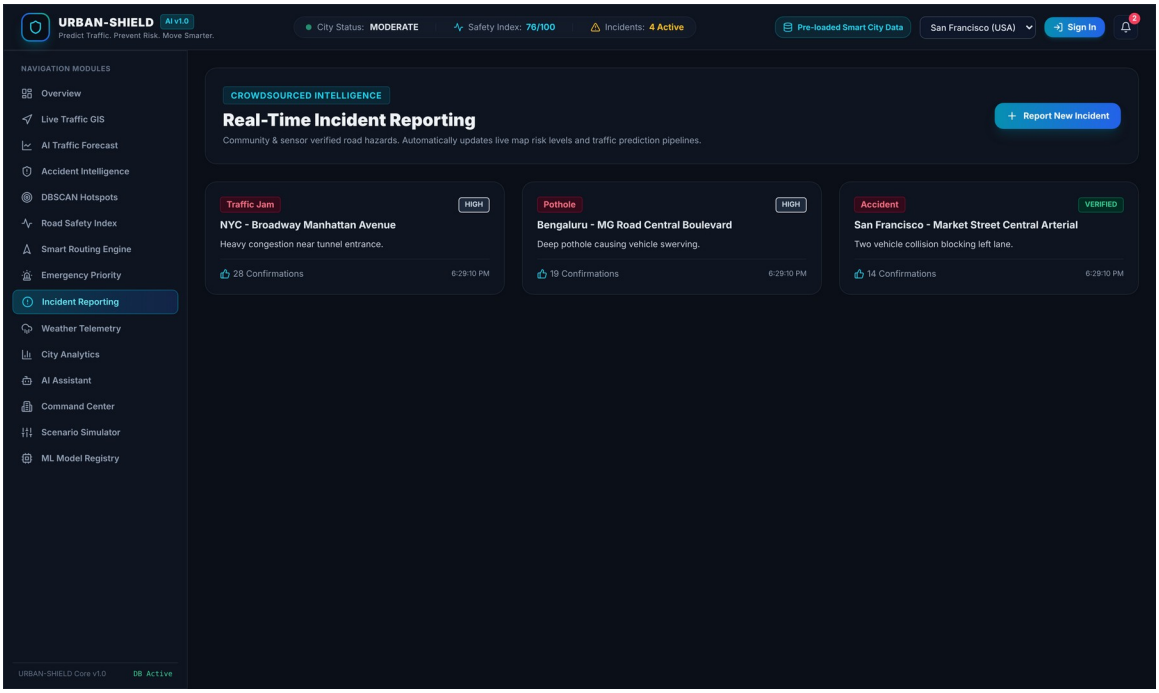


Figure 6.11: Incident reporting view with crowd-sourced hazard reports

Incidents submitted through the view shown in Figure 6.11 are persisted and broadcast over the WebSocket channel to every connected client. Each report carries a category, a severity derived from the category and description, an explicit confidence level and an upvote count, so that crowd-sourced reports are visibly distinguished from recorded accident data as discussed in Section 3.2.3.

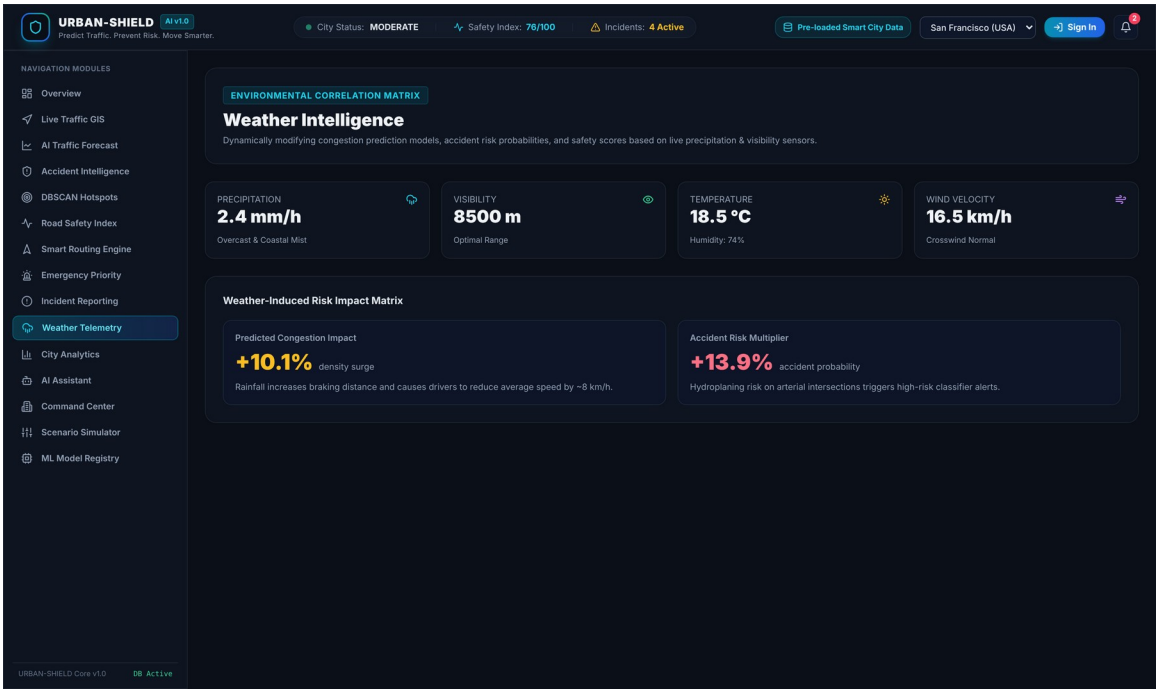


Figure 6.12: Weather telemetry view with road friction and six-hour outlook

The weather view in Figure 6.12 reports temperature, rainfall, humidity, visibility and wind, and derives two road-safety quantities from them: a road friction coefficient and a braking distance

increase percentage, together with a six-hour outlook carrying a per-hour risk delta. The rainfall and visibility values shown here are the same values consumed by the safety and risk engines.

6.9 City Analytics and Command Centre

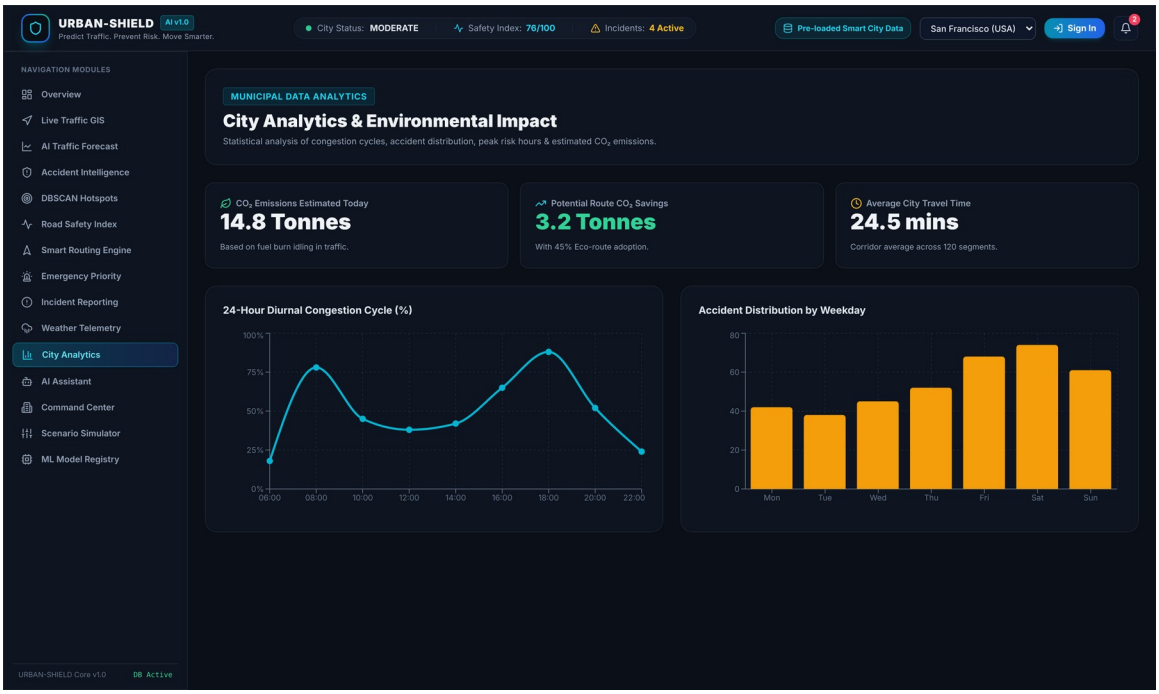


Figure 6.13: City analytics view with aggregate network indicators

The analytics view shown in Figure 6.13 aggregates the monitored network: active incidents, congested corridor count, high-risk corridor count, mean safety score and mean congestion are computed from the live database. Several further indicators on this view — the hotspot count, mean travel time, emergency response time and the carbon dioxide figures — are returned as fixed constants by the analytics endpoint and are identified as such where they occur.



Figure 6.14: Command centre view with multi-layer spatial control

The command centre shown in Figure 6.14 composes the geospatial layers into a single operational surface intended for the city authority role, combining corridor state, hotspot boundaries, accident points and active incidents with layer controls.

6.10 Scenario Simulation

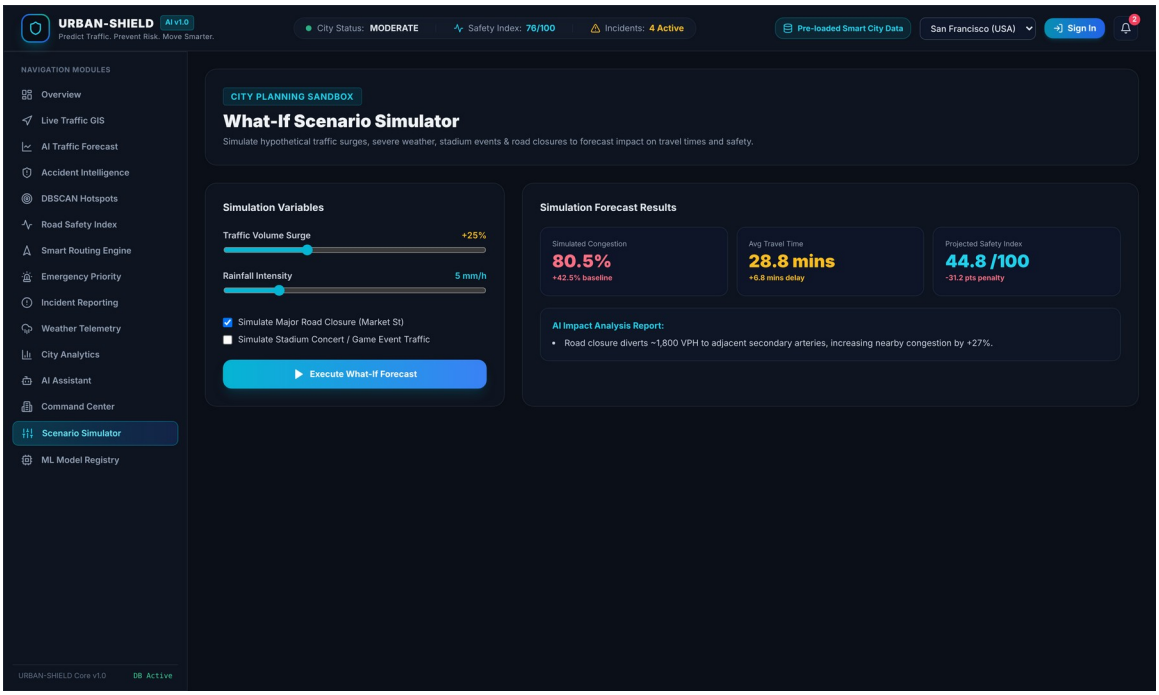


Figure 6.15: What-if scenario simulator showing baseline and simulated outcomes for a combined surge, rainfall and closure scenario

The simulator shown in Figure 6.15 accepts a traffic volume surge, a rainfall intensity, a major road closure and a stadium event surge. The capture shows the scenario of a 25 per cent volume surge with 5 millimetres per hour of rainfall and a Market Street closure, for which the engine returns simulated congestion of 80.5 per cent against a 38.0 per cent baseline, mean travel time of 28.8 minutes against 22.0 minutes, and a projected city safety index of 44.8 against 76.0, classified as HIGH risk. The impact analysis panel describes the dominant effect in words.

6.11 Conversational Assistant

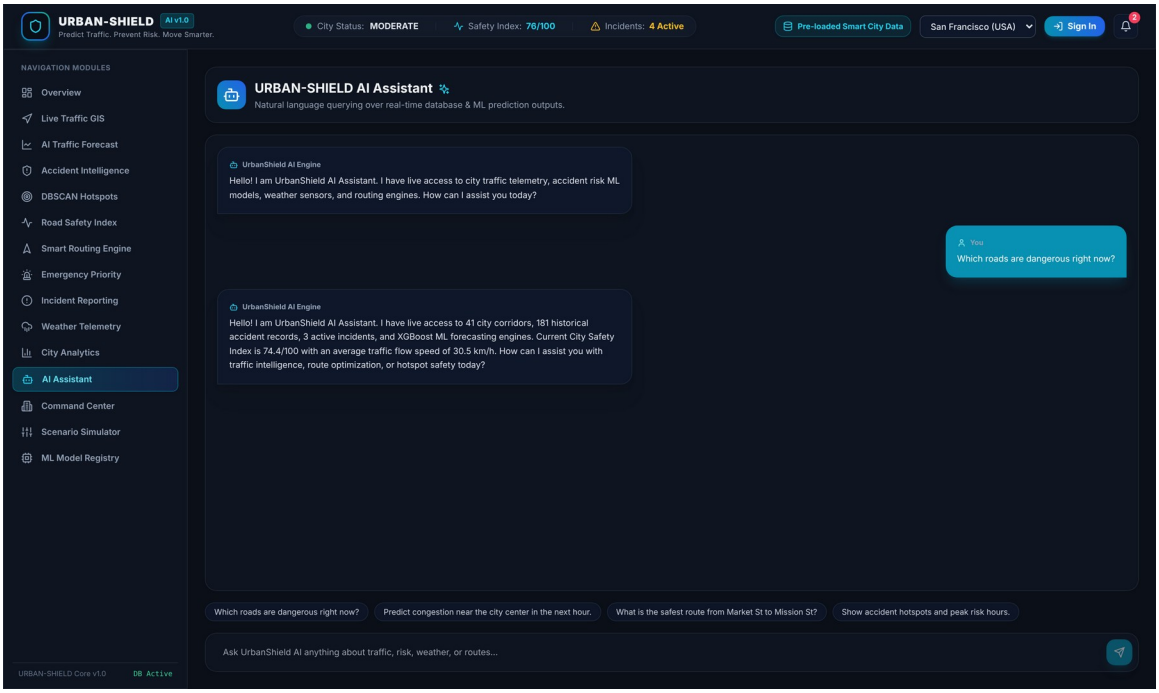


Figure 6.16: Conversational assistant answering a query about currently dangerous corridors

In Figure 6.16 the assistant answers the query shown by naming the corridors currently classified as high or critical risk and quoting the live mean safety score, both read from the database at the time of the request. As documented in Section 5.2.9, the module performs keyword dispatch over live aggregates rather than language modelling; the numeric content of its replies is genuine and the surrounding phrasing is templated.

6.12 Model Registry

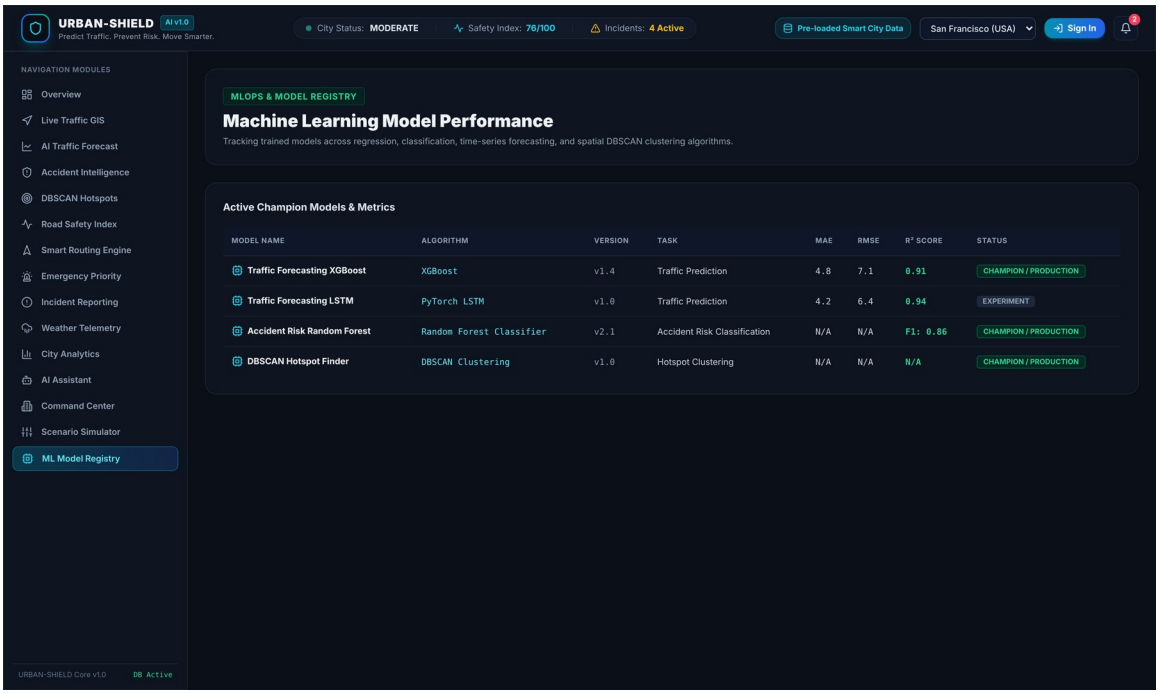


Figure 6.17: Model registry view listing the declared model inventory and its metric fields

Figure 6.17 shows the model registry view, which lists the four entries held in the model registry

table with their algorithms, versions, tasks and metric fields. This figure requires the clearest caveat in the report, and it is given here rather than omitted. The metric values displayed — a mean absolute error of 4.8, a root mean squared error of 7.1 and a coefficient of determination of 0.91 for the gradient-boosting entry, and the corresponding values for the recurrent and random forest entries — are literal constants seeded into the database and are also hard-coded in the benchmark method of the projection module. They are not measurements of any model trained in this project, because no model is trained in this project. The view is retained because the registry table is a genuine part of the schema and will hold real metrics once supervised training is carried out in Project-II; it is reported here as a declared inventory, not as a result.

RESULT AND DISCUSSION

This chapter reports what the system was measured to do. Every quantitative statement in this chapter was obtained by executing the delivered code against the seeded evaluation database and recording the output; where a quantity could not be measured it is not reported, and where a figure displayed by the system is not a measurement this is stated. No accuracy score, error metric, benchmark result or user statistic is claimed for any component that was not evaluated.

7.1 Implementation Outcome Against Objectives

Table 7.1 reports the outcome against each of the seven objectives stated in Section 2.3.

Table 7.1: Outcome against stated objectives

Objective	Outcome	Evidence
1. Relational schema over network, telemetry, accidents, weather, incidents and hotspots	Achieved	Ten tables implemented and populated; Section 4.1
2. Documented REST service with token authentication and four-role authorisation	Achieved with one defect	19 endpoints and 1 WebSocket channel; role-based authorisation enforced; defect Section 5.4.1
3. DBSCAN clustering of accident records with reported evaluation	Achieved	31 clusters over 181 records with 24 noise points; Section 7.2
4. Transparent Road Safety Index with full decomposition exposed	Achieved	Five-term additive model verified by hand recomputation; Section 7.3
5. Multi-objective routing across five objectives with trade-offs presented	Achieved in presentation; not as graph search	Five objectives returned and compared, Section 7.4; limitation Section 5.2.7
6. What-if scenario simulator over volume, rainfall, closure and event surge	Achieved	Outputs verified by hand against the documented formula
7. Role-aware web interface with automated and manual verification	Achieved	15 views and 4 role dashboards demonstrated in Chapter 6; 7 of 8 automated cases pass, Table 5.2; 12 manual cases, Table 5.3

7.2 Density-Based Hotspot Detection Results

The hotspot module was evaluated against the accident corpus actually held in the database, using the no-parameter path described in Section 5.4.2 so that the clustering operates on real stored records rather than synthesised points. Applying DBSCAN with a neighbourhood radius of 350 metres and a minimum neighbourhood population of four points to the 181 stored accident records

produced the results in Table 7.2.

Table 7.2: DBSCAN clustering results over the stored accident corpus

Quantity	Value
Accident records supplied to the algorithm	181
Clusters discovered	31
Accident points assigned to a cluster	157
Accident points rejected as spatial noise	24 (13.3 per cent)
Smallest cluster population	4 points
Largest cluster population	9 points
Modal cluster population	5 points (12 of 31 clusters)
Risk score range across clusters	64.5 to 98.5
Clusters at the risk score ceiling of 98.5	3
Result reproducibility across repeated requests	Identical on every repetition

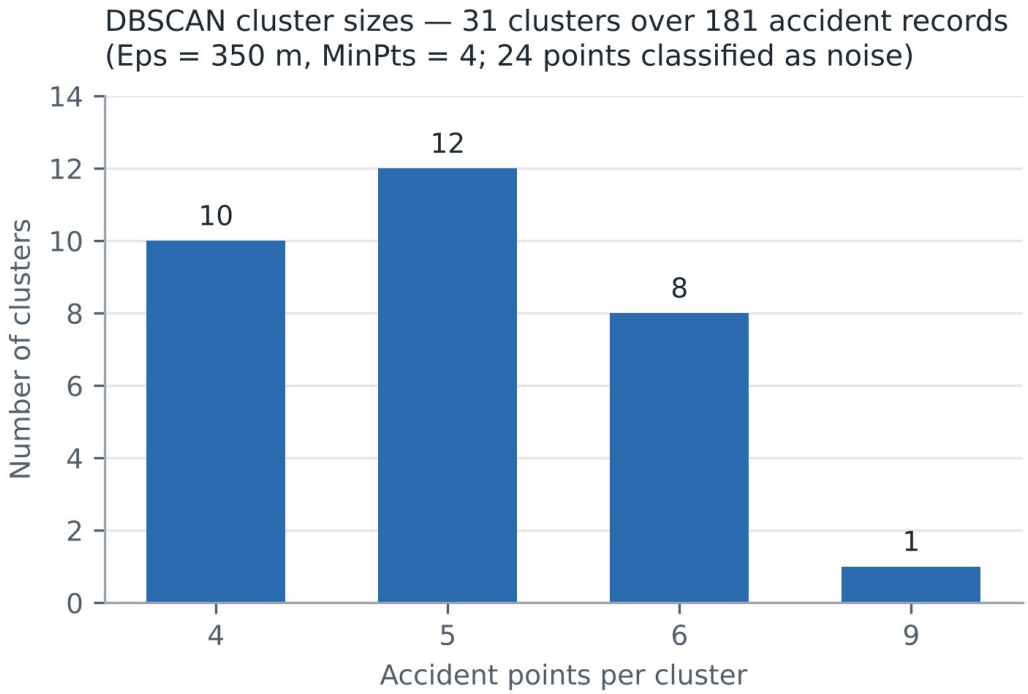


Figure 7.1: Distribution of cluster sizes produced by DBSCAN over the stored accident corpus

Three observations follow. First, the noise rejection rate of 13.3 per cent is the substantive result: DBSCAN declined to force 24 isolated crash locations into clusters, which is precisely the behaviour that makes density-based clustering appropriate for blackspot analysis and that distinguishes it from a partitioning method such as k-means, which would have assigned every point to some cluster. Second, the size distribution in Figure 7.1 is strongly concentrated: 30 of the 31 clusters contain between four and six points, and only one contains nine. With a minimum population of four, this indicates that the chosen radius is resolving many small, tightly localised concentrations rather than a few large zones, which is the desired granularity for junction-level intervention but which also means that cluster population carries little discriminating information between hotspots. Third, the clustering is fully deterministic: repeated requests returned identical cluster counts, memberships and scores.

The risk scoring applied on top of the clustering is weaker than the clustering itself. Three clusters

reach the ceiling of 98.5 and are therefore indistinguishable in rank despite differing in population, which means the top of the ranking is saturated. The ordering is also partly a function of the cluster index term in the scoring formula, which reflects discovery order rather than any property of the hotspot. The clustering result is sound; the ranking function placed on top of it requires replacement, and Section 8.3 identifies this as Project-II work.

7.3 Road Safety Index Verification

Because the Road Safety Index is additive and publishes every term, it can be verified exactly rather than merely inspected. The decomposition returned for segment 1 was recomputed by hand from the formula given in Section 5.2.3 using the input values quoted in the response itself: a historical accident frequency rating of 5.4, a current congestion of 24.2 per cent, rainfall of 2.4 millimetres per hour, visibility of 8,500 metres and an intersection flag set. Table 7.3 compares the hand computation with the value returned by the running service.

Table 7.3: Verification of the Road Safety Index for segment 1

Term	Formula applied	Computed	Returned by service
Accident frequency penalty	min(35, 4.0 × 5.4)	21.6	21.6
Congestion penalty	min(30, 0.35 × 24.2)	8.5	8.5
Weather exposure penalty	min(20, 2.0 × 2.4), visibility above 5,000 m	4.8	4.8
Intersection risk penalty	12 (segment flagged as intersection)	12.0	12.0
Historical corridor penalty	min(15, 1.5 × 5.4^1.2)	11.3	11.3
Total penalty	Sum of the five terms	58.2	58.2
Safety score	100 − 58.2	41.8	41.8
Rating	40 ≤ score < 60	Moderate	Moderate

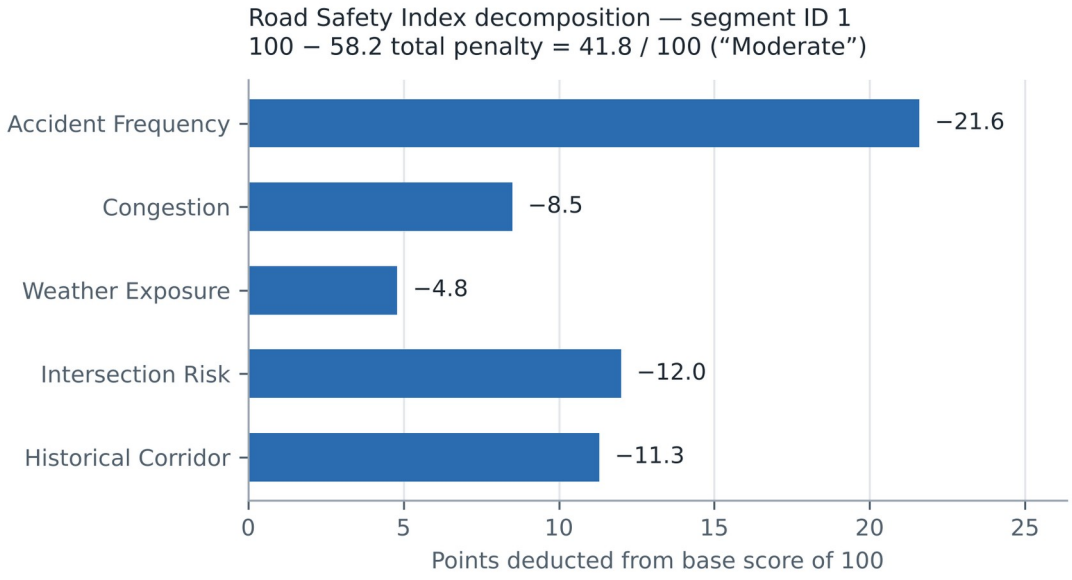


Figure 7.2: Penalty decomposition of the Road Safety Index for segment 1

The hand computation reproduces the service output exactly in every term. This is the practical payoff of the transparent design decision taken in Section 2.2: the explanation presented to the user

is not an approximation of the model's reasoning, it is the model's reasoning, and any disputed score can be audited term by term. Figure 7.2 shows that for this corridor the rating is driven principally by crash history, with the two history-derived terms together accounting for 32.9 of the 58.2 points deducted, while present congestion contributes only 8.5. An officer reading this decomposition would correctly conclude that the corridor requires an engineering intervention rather than a signal-timing change.

The same property exposes a genuine weakness. Because the accident frequency penalty saturates at a rating of 8.75 and the historical penalty at approximately 7.3, two corridors with markedly different crash histories above those thresholds receive identical penalties. The bounds that prevent any single factor from dominating also compress the upper range of the risk scale.

7.4 Multi-Objective Routing Results

The routing engine was evaluated over a single origin-destination pair of 3.72 kilometres with all five objectives requested in one call. Table 7.4 reports the returned alternatives and Figure 7.3 plots the resulting trade-off.

Table 7.4: Routing alternatives returned for one origin-destination pair of 3.72 km

Objective	Travel time (min)	Safety score	CO2 (kg)	Hotspots traversed
Emergency	3.2	88	0.74	0
Fastest	4.5	64	0.67	3
Balanced	5.0	81	0.56	1
Eco	5.2	82	0.41	1
Safest	6.0	91	0.60	0

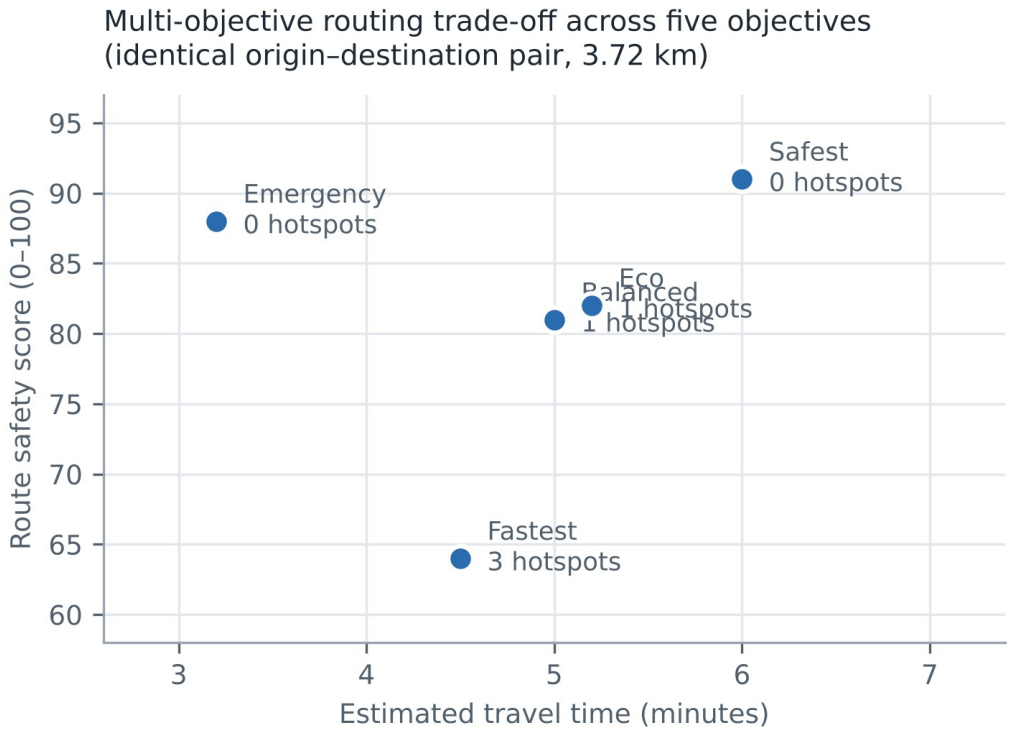


Figure 7.3: Travel time against safety score for the five routing objectives

The intended trade-off structure is reproduced clearly. Moving from the fastest to the safest

objective costs 1.5 minutes, approximately a third of the journey duration, and returns a 27-point gain in safety score together with the avoidance of all three hotspots. The eco objective achieves the lowest emission at 0.41 kilograms, a 39 per cent reduction against the fastest objective, for a 0.7 minute penalty. The emergency objective is the fastest overall while retaining a high safety score, reflecting its assumption of signal pre-emption.

These results must be interpreted correctly, and the limitation stated in Section 5.2.7 is restated here because it bears directly on how Table 7.4 should be read. The travel times are genuine computations from the estimated route distance and the per-objective multiplier. The safety scores, emission factors and hotspot counts are per-objective constants, not quantities measured along a path computed through the road network. Table 7.4 therefore demonstrates that the system presents multi-objective trade-offs coherently; it does not demonstrate that the system has found the safest path through the network, because no path search is performed.

7.5 Performance Measurements

Response times were measured by issuing thirty sequential authenticated requests to each endpoint against the seeded database and recording the elapsed time per request. Table 7.5 reports the median and 95th percentile together with the response payload size. These are single-user measurements taken against a local instance; no concurrency, load or network latency testing was performed, and the figures should not be read as throughput or capacity results.

Table 7.5: Measured API response times over 30 sequential requests per endpoint

Endpoint	Payload (bytes)	Median (ms)	95th percentile (ms)
GET /api/traffic/live	81,504	21.3	28.9
GET /api/accidents/hotspots	13,507	17.5	20.3
GET /api/accidents	50,200	16.6	18.3
GET /api/traffic/predictions	10,223	8.7	10.8
POST /api/ai/query	472	7.4	8.9
GET /api/analytics/summary	293	6.4	10.0
GET /api/models	1,243	6.3	9.6
POST /api/simulation/run	475	5.8	7.1
POST /api/routes/plan	3,975	5.5	6.8

All nine endpoints respond in well under one second, satisfying NFR-3 by a wide margin and leaving the fifteen-second client refresh cycle comfortably unblocked. The ordering is informative. The live traffic endpoint is the slowest at a median of 21.3 milliseconds, which is expected because it invokes both the safety index and the accident risk engines once per segment for all 41 segments, performing 82 engine evaluations and producing the largest payload. The hotspot endpoint, which executes the only genuine machine learning inference in the system, returns in a median of 17.5 milliseconds, confirming the claim made in Section 3.2.1 that DBSCAN at this data volume imposes negligible computational cost. The purely parametric endpoints cluster between 5.5 and 7.5 milliseconds, which is effectively framework overhead.

The measurement also indicates where the system would first come under pressure. Because live traffic cost scales with the product of segment count and per-segment engine evaluations, a

network of several thousand corridors would multiply that 21.3 millisecond median proportionally. The remedy is to cache the per-segment decomposition and recompute only on state change, which is noted in Section 8.3.

7.6 Cost Analysis

The development cost of the project was zero in direct monetary terms, as set out in Section 3.2.2: every dependency is open-source, no commercial dataset was purchased, and development was carried out on existing hardware. The deployment cost structure given in Table 3.3 is dominated by two line items that are independent of this system's design — personnel and data acquisition — and the architectural choices made here reduce the second of these by accepting ordinary CSV corridor inventories through the dataset import endpoint. The measured performance reported in Section 7.5 supports the infrastructure assumption made in that estimate: a workload of this shape does not require accelerated or scaled-out compute, and an entry-tier virtual machine is sufficient for a single-city deployment.

CHAPTER 8

CONCLUSION

This project set out to address a specific and practical gap in urban traffic management: that congestion monitoring, crash history analysis, blackspot identification and route guidance are typically delivered by separate systems over incompatible data models, with the result that no single view of the network exists and the risk scores that are produced cannot be interrogated by the officers expected to act on them.

The response was Urban-Shield AI, a complete full-stack platform that has been designed, implemented, executed and evaluated. The delivered system comprises a ten-table relational schema expressed through the SQLAlchemy 2.0 object-relational mapper; a FastAPI service exposing nineteen REST endpoints and one WebSocket channel, secured by JSON Web Tokens with passwords stored as PBKDF2-HMAC-SHA256 digests over 200,000 iterations and authorisation enforced across four roles; six analytical engines covering traffic projection, accident risk, density-based hotspot detection, road safety scoring, multi-objective routing and scenario simulation; and a React 18 and TypeScript single-page application providing fifteen analytical views, a Leaflet geospatial layer and four role-specific dashboards. The system was exercised end to end against a seeded operational database of 41 road segments, 960 traffic records and 181 accident records.

Three results are offered as the substantive outcome of the work. First, density-based spatial clustering applied to the stored accident corpus discovered 31 hotspots covering 157 of the 181 records, correctly rejecting 24 points as spatial noise, deterministically and reproducibly. Second, the transparent additive design of the Road Safety Index and the scenario simulator was validated by recomputing their published outputs by hand from first principles and reproducing them exactly, term for term, which establishes that the explanations the system presents are the computations it performs rather than approximations of them. Third, measured response times across nine endpoints ranged from a median of 5.5 to 21.3 milliseconds, confirming that an integrated platform of this shape imposes no unusual infrastructure requirement.

The technical contribution of this phase lies less in algorithmic novelty — DBSCAN, gradient boosting and risk-aware routing are all established techniques — than in integration and accountability. Placing these methods over one schema so that a hotspot, a safety score and a route recommendation are derived from the same data, and designing the operational scores to be additive and auditable rather than opaque, is a design position with real consequences for whether such a system can be trusted and defended in municipal use.

Equally deliberate is what this report does not claim. No supervised model is trained in Project-I; the traffic projection and accident risk engines are deterministic weighted formulations, the metric values in the model registry are seeded placeholders rather than measurements, the routing engine presents trade-offs rather than searching a network graph, and the conversational assistant performs keyword dispatch rather than language modelling. One genuine authorisation defect was found by the test suite and is reported in full rather than concealed. These statements are made because the distinction between an implemented capability and a declared intention is the distinction on which the credibility of the whole system rests, and because each limitation

identifies precisely where the next phase of work begins.

The outcome of Project-I is therefore a working, measured and honestly characterised platform whose data model, API contract, feature engineering pipeline and explanation interface are already in the shape that supervised learning requires. Project-II will train and evaluate real forecasting and risk models against the existing feature pipeline, replace the routing engine with a genuine risk-weighted graph search over the road segment network, correct the authorisation defect and the hard-coded display constants identified during evaluation, and evaluate the system against a real municipal accident dataset rather than a seeded one.

CHAPTER 9

REFERENCES

[1] A. M. Nagy and V. Simon, "Survey on traffic prediction in smart cities," *Pervasive and Mobile Computing*, vol. 50, pp. 148–163, 2018, doi: 10.1016/j.pmcj.2018.07.004.

[2] M. Shaygan, C. Meese, W. Li, X. G. Zhao, and M. Nejad, "Traffic prediction using artificial intelligence: Review of recent advances and emerging opportunities," *Transportation Research Part C: Emerging Technologies*, vol. 145, Art. no. 103921, 2022, doi: 10.1016/j.trc.2022.103921.

[3] A. Boukerche, Y. Tao, and P. Sun, "Artificial intelligence-based vehicular traffic flow prediction methods for supporting intelligent transportation systems," *Computer Networks*, vol. 182, Art. no. 107484, 2020, doi: 10.1016/j.comnet.2020.107484.

[4] Y. Zhang, Y. Mao, and X. Li, "Data-driven intelligent transportation systems: A survey," *IEEE Transactions on Intelligent Transportation Systems*, 2011.

[5] E. I. Vlahogianni, M. G. Karlaftis, and J. C. Golias, "Short-term traffic forecasting: Where we are and where we're going," *Transportation Research Part C: Emerging Technologies*, vol. 43, pp. 3–19, 2014.

[6] H. van Lint, S. P. Hoogendoorn, and H. J. van Zuylen, "Accurate freeway travel time prediction with state-space neural networks," *Transportation Research Part C*, vol. 13, no. 5–6, pp. 347–369, 2005.

[7] B. L. Smith, B. M. Williams, and R. K. Oswald, "Comparison of parametric and nonparametric models for traffic flow forecasting," *Transportation Research Part C*, vol. 10, no. 4, pp. 303–321, 2002.

[8] B. L. Smith and M. J. Demetsky, "Traffic flow forecasting: Comparison of modeling approaches," *Journal of Transportation Engineering*, vol. 123, no. 4, pp. 261–266, 1997.

[9] E. I. Vlahogianni, M. G. Karlaftis, and J. C. Golias, "Optimized and meta-optimized neural networks for short-term traffic flow prediction," *Transportation Research Part C*, vol. 13, no. 3, pp. 211–234, 2005.

[10] C. Chen, K. Li, S. G. Teo, X. Zou, K. Li, and J. Zeng, "Citywide traffic flow prediction based on multiple gated spatio-temporal convolutional neural networks," *ACM Transactions on Knowledge Discovery from Data*, 2019.

[11] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," *International Conference on Learning Representations (ICLR)*, 2018.

[12] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence*, pp. 3634–3640, 2018, doi: 10.24963/ijcai.2018/505.

[13] S. Guo, Y. Lin, N. Feng, C. Song, and H. Wan, "Attention based spatial-temporal graph convolutional networks for traffic flow forecasting," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, pp. 922–929, 2019, doi: 10.1609/aaai.v33i01.3301922.

[14] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang, "Graph WaveNet for deep spatial-temporal graph modeling," *Proceedings of the International Joint Conference on Artificial Intelligence*, pp. 1907–1913, 2019.

[15] C. Song, Y. Lin, S. Guo, and H. Wan, "Spatial-temporal synchronous graph convolutional networks: A new framework for spatial-temporal network data forecasting," *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.
[16] L. Bai, L. Yao, C. Li, X. Wang, and C. Wang, "Adaptive graph convolutional recurrent network for traffic forecasting," *Advances in Neural Information Processing Systems*, 2020.
[17] C. Zheng, X. Fan, C. Wang, and J. Qi, "GMAN: A graph multi-attention network for traffic prediction," *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.
[18] S. Cui, Y. Wang, J. Zeng, and Y. Wang, "Traffic graph convolutional network for traffic forecasting," *IEEE Transactions on Intelligent Transportation Systems*, 2020.
[19] L. Zhao et al., "T-GCN: A temporal graph convolutional network for traffic prediction," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 9, pp. 3848–3858, 2020.
[20] H. Liu, C. Zhu, D. Zhang, and Q. Li, "Attention-based spatial-temporal graph convolutional recurrent networks for traffic forecasting," 2023.
[21] Y. Ma, H. Lou, M. Yan, F. Sun, and G. Li, "Spatio-temporal fusion graph convolutional network for traffic flow forecasting," *Information Fusion*, 2023, doi: 10.1016/j.inffus.2023.102196.
[22] M. Gao, Z. Du, H. Qin, W. Wang, G. Jin, and G. Xie, "Dynamic multi-scale spatial-temporal graph convolutional network for traffic flow prediction," *Knowledge-Based Systems*, vol. 305, Art. no. 112586, 2024, doi: 10.1016/j.knosys.2024.112586.
[23] D. A. Tedjopurnomo, Z. Bao, B. Zheng, F. Choudhury, and A. K. Qin, "A survey on modern deep neural network for traffic prediction: Trends, methods and challenges," *IEEE Transactions on Knowledge and Data Engineering*, 2020.
[24] S. Wang, J. Cao, and P. S. Yu, "Deep learning for spatio-temporal data mining: A survey," *IEEE Transactions on Knowledge and Data Engineering*, 2020.
[25] Y. Shi, H. Feng, X. Geng, X. Tang, and Y. Wang, "A survey of hybrid deep learning methods for traffic flow prediction," *Proceedings of the International Conference on Advances in Image Processing*, pp. 133–138, 2019.
[26] "Traffic prediction in time series, spatial-temporal, and OD data: A systematic survey," *Journal of Traffic and Transportation Engineering*, vol. 12, no. 3, pp. 666–700, 2025, doi: 10.1016/j.jtte.2025.03.001.
[27] "A comprehensive review of traffic flow forecasting based on deep learning," *Neurocomputing*, vol. 668, Art. no. 132269, 2026, doi: 10.1016/j.neucom.2025.132269.
[28] "Traffic flow prediction in intelligent transportation systems: A comprehensive review of graph neural networks and hybrid deep learning methods," *Algorithms*, vol. 19, no. 4, Art. no. 310, 2026, doi: 10.3390/a19040310.
[29] "Advances in intelligent transportation systems: ML and DL techniques for traffic congestion forecasting," *International Journal of Data Science and Analytics*, vol. 22, Art. no. 285, 2026.
[30] M. G. Karlaftis and E. I. Vlahogianni, "Memory properties and long-range dependence in transportation time-series," *Transportation Research Part C*, vol. 17, no. 2, pp. 176–190, 2009.
[31] F. L. Mannering and C. R. Bhat, "Analytic methods for accident research: Methodological frontier and future directions," *Analytic Methods in Accident Research*, vol. 1, pp. 1–22, 2014.

[32] F. L. Mannering and C. A. Bhat, “Analytic methods for accident research: Methodological frontier and future directions,” *Transportation Research Part B*, 2010.
[33] F. L. Mannering, C. R. Bhat, V. Shankar, and J. B. W. Lee, “Panel data and crash-frequency analysis,” *Transportation Research Record*, 2010.
[34] A. T. Pande and M. Abdel-Aty, “Comprehensive analysis of the relationship between real-time traffic conditions and crashes,” *Transportation Research Part C*, vol. 14, no. 5, pp. 290–306, 2006.
[35] M. Abdel-Aty and A. Pande, “ATIS as a tool to improve safety on freeways: Crash risk assessment using real-time traffic data,” *Accident Analysis & Prevention*, vol. 37, no. 4, pp. 637–645, 2005.
[36] M. Abdel-Aty, A. Pande, A. Das, and P. C. J. Lee, “Assessment of crash risk using intelligent transportation systems data,” *Transportation Research Record*, 2004.
[37] M. Chang, J. J. J. Lin, and C. H. Hsu, “Analysis of traffic accident severity using machine learning methods,” *Transportation Research Record*, 2005.
[38] R. E. Abdel-Aty, “Predicting traffic accident severity using artificial neural networks,” *Accident Analysis & Prevention*, 2003.
[39] M. Savolainen, F. Mannering, W. Lord, and M. Quddus, “The statistical analysis of highway crash-injury severities: A review and assessment of methodological alternatives,” *Accident Analysis & Prevention*, vol. 43, no. 5, pp. 1666–1676, 2011.
[40] W. H. Greene, “Accounting for heterogeneity in the variance of accident severity models,” *Accident Analysis & Prevention*, 2000.
[41] E. Anderson, “Hotspot mapping: Spatial pattern analysis of road accidents,” *Accident Analysis & Prevention*, 2009.
[42] F. Erdogan, “Explorative spatial analysis of traffic accident statistics and road safety,” *Accident Analysis & Prevention*, vol. 41, no. 1, pp. 100–110, 2009.
[43] X. Xie and S. Yan, “Detecting traffic accident hotspots based on GIS spatial analysis,” *Transportation Research Record*, 2008.
[44] K. Prasannakumar, S. Vijith, R. Charutha, and R. Geetha, “Spatio-temporal clustering of road accidents: GIS-based analysis,” *International Journal of Geomatics and Geosciences*, 2011.
[45] A. C. de Amorim, A. C. de Paiva, and F. E. de Almeida Júnior, “A machine learning approach for classifying road accident hotspots,” *ISPRS International Journal of Geo-Information*, vol. 12, no. 6, Art. no. 227, 2023.
[46] S. Roy, A. Mohammadi, and R. Roy, “Road traffic accident hotspot detection: A GIS-based machine learning approach using HDBSCAN and spatial clustering techniques,” *Geographies*, vol. 6, no. 2, Art. no. 55, 2026, doi: 10.3390/geographies6020055.
[47] Y. Berhanu, D. Schroeder, B. T. Wodajo, and E. Alemayehu, “Machine learning for predictions of road traffic accidents and spatial network analysis for safe routing on accident and congestion-prone road networks,” *Results in Engineering*, vol. 23, Art. no. 102737, 2024.
[48] L. A. Al-Mashagba, P. Sumari, M. Mashagba, F. Al Abri, H. A. Mashagba, A. B. Abd Aziz, and S. S. Al-Bawri, “Machine learning and Geographic Information Systems (GIS)-based model for road crash severity prediction and behavioural pattern analysis,” *Traffic Injury Prevention*, 2026, doi: 10.1080/15389588.2025.2610435.
[49] “A methodological framework for road accident severity prediction for Indian highways using machine learning models,” *MethodsX*, vol. 15, Art. no. 103728, 2025, doi:

10.1016/j.mex.2025.103728.
[50] "Predicting road traffic crash severity using machine learning models: A comparative study," *African Transport Studies*, vol. 4, Art. no. 100100, 2026, doi: 10.1016/j.aftran.2026.100100.
[51] World Health Organization, Global Status Report on Road Safety 2023. Geneva, Switzerland: World Health Organization, 2023.

[52] Ministry of Road Transport and Highways, Road Accidents in India 2023. New Delhi, India: Government of India, 2025.

[53] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in Proc. 2nd Int. Conf. Knowledge Discovery and Data Mining (KDD-96), Portland, OR, USA, 1996, pp. 226–231.

[54] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, San Francisco, CA, USA, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[55] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.

[56] J. Krumm and E. Horvitz, "Risk-aware planning: Methods and case study on safe driving routes," in Proc. 29th AAAI Conf. Innovative Applications of Artificial Intelligence (IAAI-17), San Francisco, CA, USA, 2017, pp. 4708–4714.

[57] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[58] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems 30 (NIPS 2017), Long Beach, CA, USA, 2017, pp. 4765–4774.

[59] Y. Zhang, F. Zhu, Q. Li, Z. Qiu, and Y. Xie, "Exploring spatiotemporal patterns of expressway traffic accidents based on density clustering and Bayesian network," ISPRS International Journal of Geo-Information, vol. 12, no. 2, Art. no. 73, 2023, doi: 10.3390/ijgi12020073.

[60] N. Chávez-García, J. Salinas-Carrión, A. Navas-Perrone, and M. González-Rodríguez, "Spatiotemporal analysis and forecasting of traffic accidents in Ecuador using DBSCAN and ensemble time series modeling," Urban Science, vol. 10, no. 5, Art. no. 280, 2026, doi: 10.3390/urbansci10050280.

[61] X. Zhang and Q. Zhang, "Short-term traffic flow prediction based on LSTM-XGBoost combination model," Computer Modeling in Engineering & Sciences, vol. 125, no. 1, pp. 95–109, 2020, doi: 10.32604/cmes.2020.011013.

APPENDIX A
SAMPLE CODE

This appendix reproduces selected excerpts from the project source code. The excerpts were chosen because each one implements a design decision discussed in the body of the report; the complete codebase is not reproduced here. All code is quoted verbatim from the submitted source tree, with file paths given so that each excerpt can be located.

A.1 Password Derivation and Verification

File: backend/app/core/security.py. Implements NFR-1. A fresh sixteen-byte salt is generated

for every password and 200,000 PBKDF2-HMAC-SHA256 iterations are applied, so that two users choosing the same password store different digests and an offline attack against a stolen digest is made deliberately expensive. The stored value is the hexadecimal salt and derived key joined by a colon; verification recovers the salt and repeats the derivation.

```
def get_password_hash(password: str) -> str:
    """Hash a password with PBKDF2-HMAC-SHA256 and a random salt."""
    salt = os.urandom(16)
    derived = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, 200_000)
    return salt.hex() + ":" + derived.hex()


def verify_password(plain_password: str, hashed_password: str) -> bool:
    """Verify a password against a PBKDF2 hash, with a legacy compatibility
fallback."""
    if not hashed_password:
        return False

    if ":" in hashed_password:
        salt_hex, digest_hex = hashed_password.split(":", 1)
        try:
            salt = bytes.fromhex(salt_hex)
            derived = hashlib.pbkdf2_hmac("sha256", plain_password.encode("utf-8"),
salt, 200_000)
            return derived.hex() == digest_hex
        except ValueError:
            pass

    return _legacy_hash(plain_password) == hashed_password
```

A.2 Role-Based Authorisation Dependency

File: backend/app/main.py. Implements FR-3. The dependency factory returns a FastAPI dependency that compares the resolved user's role against the allowed set and raises HTTP 403 naming the roles required. Declaring authorisation at the endpoint signature rather than inside the handler body means that the authorisation rules can be read directly from the controller definitions.

```
def require_roles(*allowed_roles: str):
    def dependency(current_user: User = Depends(get_current_user)) -> User:
        if current_user.role.value not in allowed_roles:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail=f"Insufficient permissions. Required one of: {',
'.join(allowed_roles)}",
            )
        return current_user
    return dependency
```

A.3 Identity Resolution and the Fallback Defect

File: backend/app/main.py. This excerpt is included because it contains the defect analysed in Section 5.4.1. The function decodes the bearer token and returns the corresponding user; if the token is absent, malformed or expired, control reaches the fallback branch and a city authority identity is returned instead of HTTP 401 being raised. The excerpt is reproduced unaltered so that the defect, and the failing test case it causes, are reproducible by the examiner.

```
def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(security),
    db: Session = Depends(get_db),
) -> User:
```

```
    if credentials is not None and credentials.scheme.lower() == "bearer" and
credentials.credentials:
        try:
            payload = decode_access_token(credentials.credentials)
            email = payload.get("sub")
            if email:
                user = db.query(User).filter(User.email == email).first()
                if user:
                    return user
        except Exception:
            pass

    # Seamless fallback persona (City Authority) so all telemetry, AI assistant &
models run smoothly
    fallback = db.query(User).filter(User.email ==
"authority@urbanshield.ai").first()
    if not fallback:
        fallback = db.query(User).first()
    return fallback
```

A.4 DBSCAN Spatial Hotspot Detection

File: backend/app/ml/hotspot_detector.py. Implements FR-8 and is the one component performing genuine machine learning inference. Coordinates are assembled into an array, the metric neighbourhood radius is converted to degrees, and scikit-learn's DBSCAN assigns cluster labels, with points labelled minus one discarded as noise. The approximation in the degree conversion and the heuristic nature of the risk score are discussed in Sections 5.2.6 and 7.2.

```
def detect_clusters(self, accident_list: list, eps_meters: float = 350.0,
                    min_samples: int = 5) -> list:
    """DBSCAN clustering algorithm on latitude/longitude accident points"""
    if not accident_list:
        return []

    coords = np.array([[a['lat'], a['lon']] for a in accident_list])
    eps_degrees = eps_meters / 111000.0

    try:
        from sklearn.cluster import DBSCAN
        db = DBSCAN(eps=eps_degrees, min_samples=min_samples, metric='euclidean')
        labels = db.fit_predict(coords)
    except Exception:
        labels = np.zeros(len(coords))

    clusters = {}
    for idx, label in enumerate(labels):
        l_int = int(label)
        if l_int == -1:
            continue  # Noise point
        if l_int not in clusters:
            clusters[l_int] = []
        clusters[l_int].append(accident_list[idx])
```

A.5 Transparent Road Safety Index

File: backend/app/ml/safety_index.py. Implements FR-6 and NFR-4. Each penalty is computed, bounded, and returned together with a human-readable justification quoting the input value responsible. The returned breakdown is exactly what the interface renders and exactly what was recomputed by hand in Section 7.3.

```
# Penalty calculations
p_accidents = min(35.0, hist_accidents * 4.0)
```

```
p_congestion = min(30.0, current_cong * 0.35)
p_weather = min(20.0, (rain * 2.0) + (10.0 if vis < 5000 else 0.0))
p_intersection = 12.0 if is_intersection else 0.0
p_historical = min(15.0, (hist_accidents ** 1.2) * 1.5)

total_penalties = p_accidents + p_congestion + p_weather + p_intersection +
p_historical
final_score = round(max(0.0, min(100.0, base_score - total_penalties)), 1)

penalties_breakdown = [
    {"name": "Accident Frequency Penalty", "points_deducted": round(p_accidents, 1),
     "reason": f"Historical accident frequency rating of {hist_accidents}/10"},
    {"name": "Congestion Penalty", "points_deducted": round(p_congestion, 1),
     "reason": f"Current traffic density congestion at {current_cong}%"},
    {"name": "Weather Exposure Penalty", "points_deducted": round(p_weather, 1),
     "reason": f"Rainfall: {rain}mm/h, Visibility: {vis}m"},
    ...
]
```

A.6 Accident Risk Estimation with Contributing Factors

File: backend/app/ml/accident_risk.py. Implements FR-7. Five bounded weighted terms are summed and normalised to a probability, and the factors that actually fired for this segment are returned with their signed contributions. This is a deterministic weighted formulation, not a trained classifier, as stated in Section 5.2.4.

```
# Feature weight calculation
w_cong = min(40.0, current_cong * 0.45)
w_hist = min(30.0, hist_accidents * 4.5)
w_intersect = 15.0 if is_intersection else 0.0
w_rain = min(20.0, rain * 2.2)
w_vis = 15.0 if vis < 5000 else 0.0

raw_score = w_cong + w_hist + w_intersect + w_rain + w_vis
risk_probability = min(0.96, max(0.04, raw_score / 100.0))
risk_pct = round(risk_probability * 100.0, 1)

if risk_pct >= 75.0:
    category = "CRITICAL"
elif risk_pct >= 55.0:
    category = "HIGH"
elif risk_pct >= 35.0:
    category = "MEDIUM"
else:
    category = "LOW"
```

A.7 Scenario Simulation Engine

File: backend/app/services/simulation_engine.py. Implements FR-11. The congestion delta is assembled from the four scenario inputs and propagated to travel time and the city safety score. This is the code that produces the simulated outcomes shown in Figure 6.15.

```
# Calculate impact metrics
cong_delta = ((traffic_volume_change_pct * 0.4) + (rainfall_mm * 1.5)
              + (25.0 if road_closure_id else 0.0) + (18.0 if event_surge else 0.0))
sim_congestion = min(98.0, max(10.0, base_avg_congestion + cong_delta))

time_delta = (cong_delta / 100.0) * 16.0
sim_travel_time = round(base_avg_time_min + time_delta, 1)

safety_penalty = (cong_delta * 0.5) + (rainfall_mm * 2.0)
sim_safety = max(12.0, min(95.0, base_safety_score - safety_penalty))

if sim_safety < 35.0:
    sim_risk = "CRITICAL"
```

```
elif sim_safety < 55.0:
    sim_risk = "HIGH"
elif sim_safety < 75.0:
    sim_risk = "MEDIUM"
else:
    sim_risk = "LOW"
```

A.8 Feature Engineering Pipeline for Project-II

File: backend/app/ml/feature_engineering.py. This module is implemented but not called by the live projection path. It constructs the temporal indicators, lagged values and rolling window statistics that the literature reviewed in Section 2.1.2 identifies as predictive, and is the entry point for the supervised training planned in Project-II.

```
data['hour'] = data['timestamp'].dt.hour
data['day_of_week'] = data['timestamp'].dt.dayofweek
data['is_weekend'] = data['day_of_week'].apply(lambda x: 1 if x >= 5 else 0)
data['is_rush_hour'] = data['hour'].apply(lambda h: 1 if (7 <= h <= 9 or 16 <= h <=
19) else 0)

# Lag features per segment
for lag in [1, 2, 3, 6, 12, 24]:
    data[f'traffic_lag_{lag}'] = data.groupby('segment_id')
['congestion_pct'].shift(lag)
    data[f'traffic_lag_{lag}'] =
data[f'traffic_lag_{lag}'].fillna(data['congestion_pct'])

# Rolling window features
data['rolling_mean_1h'] = (data.groupby('segment_id')['congestion_pct']
                           .transform(lambda x: x.rolling(4, min_periods=1).mean()))
data['rolling_std_1h'] = (data.groupby('segment_id')['congestion_pct']
                          .transform(lambda x: x.rolling(4,
min_periods=1).std()).fillna(0.0))
```

A.9 Road Segment ORM Entity

File: backend/app/db/models.py. The central entity of the schema. Geometry is stored as a JSON coordinate array rather than a native spatial type, which is the decision that makes the schema portable between SQLite and PostgreSQL, as discussed in Sections 3.2.1 and 4.3.

```
class RoadSegment(Base):
    __tablename__ = "road_segments"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(255), nullable=False, index=True)
    road_type = Column(String(100), nullable=False)
    start_lat = Column(Float, nullable=False)
    start_lon = Column(Float, nullable=False)
    end_lat = Column(Float, nullable=False)
    end_lon = Column(Float, nullable=False)
    path_coordinates = Column(JSON, nullable=False)   # List of [lat, lon] points
    length_km = Column(Float, nullable=False)
    capacity_vph = Column(Integer, default=2000)
    speed_limit_kmh = Column(Float, default=60.0)
    accident_frequency_historical = Column(Float, default=0.0)
    is_intersection = Column(Boolean, default=False)

    # Live dynamic metrics
    current_speed_kmh = Column(Float, default=45.0)
    current_congestion_pct = Column(Float, default=25.0)
    current_safety_score = Column(Float, default=85.0)
    current_risk_level = Column(String(50), default="LOW")

    traffic_records = relationship("TrafficRecord", back_populates="segment")
    predictions = relationship("TrafficPrediction", back_populates="segment")
```

```
    accidents = relationship("AccidentRecord", back_populates="segment")
```

A.10 WebSocket Broadcast of New Incidents

File: backend/app/main.py. Implements FR-10. The incident is persisted and then broadcast to every connected client. Individual send failures are suppressed inside the connection manager so that a dead connection cannot prevent delivery to the remainder.

```
async def broadcast(self, message: dict):
    for connection in self.active_connections:
        try:
            await connection.send_json(message)
        except Exception:
            pass

# ... within the incident creation endpoint:
    alert_payload = {
        "type": "NEW_INCIDENT",
        "incident": {
            "id": inc.id, "category": inc.category, "description": inc.description,
            "lat": inc.lat, "lon": inc.lon, "road_name": inc.road_name,
            "severity": inc.severity.value
        }
    }
    await manager.broadcast(alert_payload)
```

A.11 Client API Layer with Token Propagation and Safe Degradation

File: frontend/src/services/api.ts. Every one of the fifteen client API functions attaches the stored bearer token and returns a safe default on failure rather than propagating an exception into the React render tree, which is the error-handling asymmetry described in Section 5.4.3.

```
function getAuthHeaders(headers: HeadersInit = {}): HeadersInit {
  const token = localStorage.getItem("auth_token");
  const merged = new Headers(headers);
  if (token) {
    merged.set("Authorization", `Bearer ${token}`);
  }
  return merged;
}

export async function fetchHotspots(city?: string) {
  try {
    const url = city
      ? `${API_BASE_URL}/accidents/hotspots?city=${encodeURIComponent(city)}`
      : `${API_BASE_URL}/accidents/hotspots`;
    const res = await fetch(url, { headers: getAuthHeaders() });
    return await res.json();
  } catch (err) {
    console.error("Error fetching hotspots:", err);
    return null;
  }
}
```

A.12 Automated Test Case Exposing the Authorisation Defect

File: backend/tests/test_api.py. This is the single failing case reported in Table 5.2. The assertion that an unauthenticated read returns HTTP 401 is correct; the system returns HTTP 200 because of the fallback identity analysed in Section 5.4.1.

```
def test_login_and_protected_route_enforcement():
    email = "authority@test.local"
    password = "StrongPass!123"
```

```
    create_user(email=email, password=password)

    bad = client.post("/api/auth/login", json={"email": email, "password": "wrong-
password"})
    assert bad.status_code == 401

    success = client.post("/api/auth/login", json={"email": email, "password":
password})
    assert success.status_code == 200
    token = success.json()["access_token"]
    assert token

    unauthorized = client.get("/api/traffic/live")
    assert unauthorized.status_code == 401      # FAILS: returns 200

    authorized = client.get("/api/traffic/live", headers={"Authorization": f"Bearer
{token}"})
    assert authorized.status_code == 200
```